Algoritmi 2

I seguenti appunti sono a cura di ***Catalin Ceban*** *e si basano sul corso di "Algoritmi 2" erogato nell'A.A. 2026 dal professore Paul Joseph Wollan per il C.d.L Informatica (33503)*

12 novembre 2026

Indice

1 Teoria dei Grafi **2**

1.1 Il problema dei ponti di Konigsberg 3

1.2 Rappresentazione fisica dei grafi 5

1.2.1 Matrici di adiacenza 6

1.2.2 Liste di adiacenza 6

1.3 Passeggiate e cammini 8

1.4 Algoritmo di ricerca di un ciclo all'interno di un grafo 8

1.5 DFS - Depth-First-Search 12

1.6 Generici problemi su grafi 19

1.6.1 Problema di bicolorazione di un grafo 20

1.7 Lavoro su grafi orientati 23

1.7.1 Ricerca di un ordine topologico 25

1.7.2 Comportamento nei grafi non diretti 32

1.8 Ponti in un Grafo 39

1.9 Grafi e componenti fortemente connesse 43

1.10 BFS - Breadth-First-Search 55

1.10.1 Estrazione del cammino minimo 58

1.11 Grafi pesati 63

1.11.1 Calcolo delle distanze pesate 65

2 Algoritmi Greedy **69**

2.0.1 Strategie di ordinamento per l'algoritmo Greedy 69

1

Teoria dei Grafi

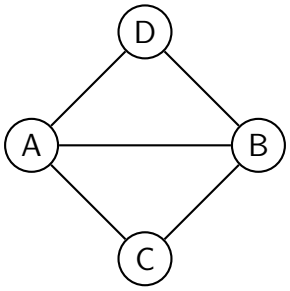
Definizione 1.0.1: Grafo

Un **grafo** è una coppia $G = (V, E)$, dove:

- V rappresenta l'insieme dei **vertici** (o nodi).
- E rappresenta l'insieme degli **archi** (o lati). Matematicamente parlando, un arco è a sua volta una coppia di valori (v_1, v_2), dove v_1, v_2 sono de vertici del grafo.

Per comodità di notazione, nel corso del manuale denoteremo con $n = |V|$ il numero dei vertici e con $m = |E|$ il numero degli archi.

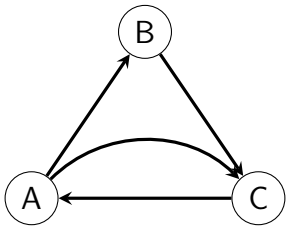
A titolo esemplificativo, consideriamo la seguente rappresentazione grafica di un grafo:



Riguardo agli archi, potremmo avere anche un orientamento.

Definizione 1.0.2: Grafo diretto (o orientato)

Un grafo si dice **diretto** o **orientato** se ciascuno dei suoi archi ha un orientamento, cioè punta specificatamente verso un dato nodo.



È importante notare che possiamo avere anche due archi sugli stessi nodi ma con orientamento diverso (come mostrato nel collegamento bidirezionale).

Definizione 1.0.3: Arco incidente e vertici adiacenti

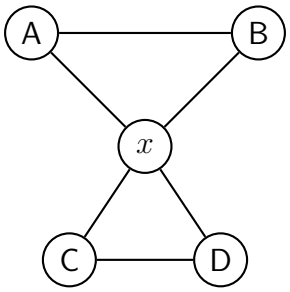
Sia f un arco che collega due vertici x e y, formalmente rappresentabile come $x \longleftarrow f \longrightarrow y$. Diciamo che:

- L'arco f è **incidente** ai vertici x e y.
- Il vertice x è **adiacente** al vertice y.

Definizione 1.0.4: Grado di un vertice

Il **grado** di un vertice x, denotato con $\deg(x)$, è dato dal numero di archi incidenti in x.

Ad esempio, considerando un nodo centrale collegato a quattro vertici periferici:



Per il nodo centrale x contrassegnato, avremo chiaramente $\deg(x) = 4$, poiché vi incidono esattamente quattro archi **distinti**.

1.1 Il problema dei ponti di Konigsberg

Il problema classico da cui nasce la teoria dei grafi è il famoso problema dei ponti di Konigsberg:

Ci si chiede: *è possibile attraversare tutti i ponti una sola volta e tornare al punto di partenza?* Come prima cosa, quando si vuole tradurre un problema in un problema di

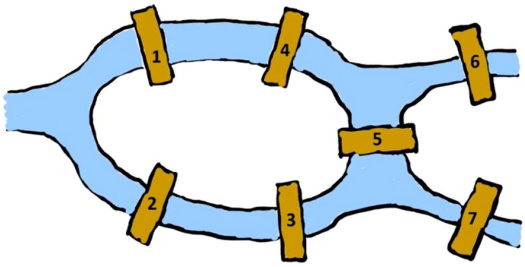


Figura 1.1: Rappresentazione grafica del problema dei ponti di Konigsberg.

teoria dei grafi, è buona cosa cercare di dare una rappresentazione grafica il più accurata possibile del problema. Essendo questo un problema di piccole dimensioni, possiamo ottenere la seguente struttura:

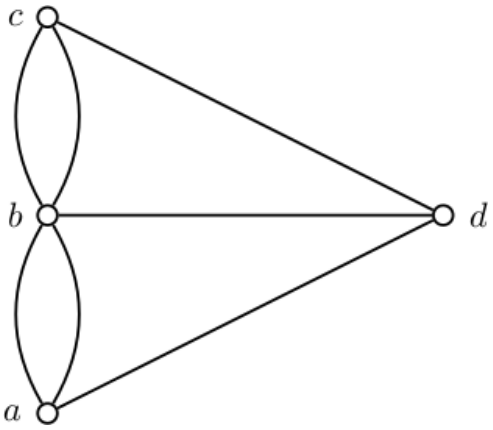


Figura 1.2: Rappresentazione a grafo del problema dei ponti.

Si invita il lettore a provare manualmente se esiste tale sequenza di attraversamenti, ma la risposta alla domanda rimane **negativa**. Prima di giustificarne la motivazione, dobbiamo fornire delle importantissime definizioni riguardanti i grafi.

Definizione 1.1.1: Passeggiata in un grafo

Una **passeggiata** (o **walk**) in un grafo è una sequenza $v_0, e_1, v_1, e_2, v_2, \ldots, v_{k-1}, e_k, v_k$ che alterna vertici e archi. Formalmente, può essere anche indicata con $W = (v_1, v_2, \ldots, v_k)$, dunque omettendo gli archi ma indicando solo in ordine di apparizione i vertici.

Definizione 1.1.2: Passeggiata chiusa

Una passeggiata si dice **chiusa** se $v_0 = v_k$, cioè il punto di inizio coincide con il punto di arrivo.

Possiamo adesso giustificare, più o meno, la risposta al problema dei ponti di Konigsberg: qualsiasi grafo con almeno un nodo x con grado dispari **non** ammette una passeggiata chiusa che utilizza tutti gli archi una sola volta.

E se invece tutti i nodi avessero grado pari? Interessante, domanda, ma **non è detto**.

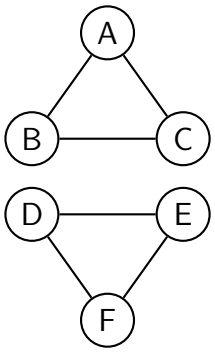


Figura 1.3: Grafo non connesso formato da due componenti triangolari disgiunte. Tutti i vertici hanno $\deg(v) = 2$, ma il grafo complessivo non è connesso e dunque non è possibile effettuare una passeggiata chiusa al suo interno.

Definizione 1.1.3: Grafo connesso

Un grafo si dice **connesso** se per ogni coppia di vertici esiste una passeggiata da x a y. In poche parole, un grafo si dice connesso se ogni coppia di vertici scelta è collegata in qualche modo.

Teorema 1.1.1: Caratterizzazione di Eulero

Esiste una passeggiata chiusa che contiene ogni arco una sola volta se e solo se:

1. Ogni vertice ha **grado pari**.
2. Il grafo è **connesso**.

1.2 Rappresentazione fisica dei grafi

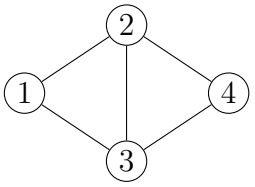
La rappresentazione informatica dei grafi in memoria è fondamentale per determinare l'efficienza degli algoritmi che operano su di essi. Esistono principalmente due modalità standard per rappresentare un grafo: le **matrici di adiacenza** e le **liste di adiacenza**.

1.2.1 Matrici di adiacenza

Si stabilisce una matrice $n \times n$ dove $n = \#V$, e dove:

$$m_{i,j} = \begin{cases} 1 & \text{se } i \text{ è adiacente a } j \\ 0 & \text{altrimenti} \end{cases}$$

Esempio:



$$\begin{array}{c} \\ 1 \\ 2 \\ 3 \\ 4 \end{array} \begin{array}{c} \begin{array}{cccc} 1 & 2 & 3 & 4 \end{array} \\ \begin{bmatrix} 0 & 1 & 1 & 0 \\ 1 & 0 & 1 & 1 \\ 1 & 1 & 0 & 1 \\ 0 & 1 & 1 & 0 \end{bmatrix} \end{array}$$

Se il grafo non è diretto, la matrice sarà simmetrica rispetto alla diagonale.

Vediamo di seguito il **costo computazionale** di alcune delle operazioni elementari che possono essere effettuate sui grafi:

- **Visita del grafo e memorizzazione**: $\boldsymbol{\Theta(n^2)}$, in quanto viene memorizzato come una matrice $n \times n$.
- Per verificare se **i è adiacente a j**: $\boldsymbol{\Theta(1)}$, in quanto è sufficiente controllare il valore di $M[i][j]$: se esso corrisponde ad 1 allora i due nodi sono adiacenti, altrimenti no.
- **Verificare il grado di un dato vertice i**: $\boldsymbol{\Theta(n)}$, in quanto occorre scorrere la riga i-esima della matrice e contare quanti 1 ci sono.

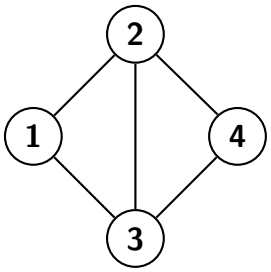
1.2.2 Liste di adiacenza

Una lista di adiacenza è composta da:

- Un array in cui ogni elemento è un nodo del grafo.
- Per ogni nodo del grafo, dunque per ogni elemento dell'array, viene associato anche un puntatore `neigbh` che punta ad una lista concatenata contenente tutti i nodi adiacenti al nodo corrente.

Costo computazionale delle operazioni elementari:

- Per visitare e memorizzare: $\mathcal{O}\left(n + \sum_v \deg(v)\right) = \mathcal{O}(n + 2m) = \mathcal{O}(n + m)$.
  Spieghiamo il perché di questo costo computazionale:



$$\begin{bmatrix} 1 \\ 2 \\ 3 \\ 4 \end{bmatrix} \begin{array}{l} \longrightarrow [2,3] \\ \longrightarrow [1,3,4] \\ \longrightarrow [1,2,4] \\ \longrightarrow [2,3] \end{array}$$

1. Sicuramente, per memorizzare una lista di adiacenza abbiamo bisogno innanzitutto di un array che tenga traccia di tutti i nodi. Ecco da dove viene il $\mathcal{O}(n)$.
2. Inoltre, per ogni nodo dobbiamo memorizzare anche tutti i nodi a lui adiacenti. Il numero di nodi adiacenti corrisponde al **grado** del singolo nodo, ossia al numero di archi incidenti al nodo. Sappiamo che il numero di archi è quantificabile con m, ma in questo caso stiamo contando ogni arco ben due volte (vedendo anche l'esempio, nel nodo 1 stiamo dicendo che è adiacente a 2, e nel nodo due stiamo dicendo che è adiacente ad 1). Dovremo dunque contare due volte ogni arco, ottenendo una quantità pari a $2m$. Il costo computazionale diventa dunque nell'ordine di $n+2m$, ma sapendo che le costanti moltiplicative sono indifferenti asintoticamente, tale valore diventa semplicemente $O(n+m)$.

- Per verificare se i è adiacente a j: $\mathcal{O}(\deg(i))$. Questo in quanto come vediamo la lista di adiacenza relativa ad ogni nodo ha un numero di elementi pari al grado di quel nodo. Dunque per verificare se due nodi sono adiacenti o meno è necessario scorrere la lista di adiacenza di quel nodo e cercare il nodo adiacente interessato.

Per quanto riguarda la memorizzazione e la visita completa di un grafo memorizzato mediante liste di adiacenza, si potrebbe fare un appunto riguardo ai grafi diretti. In tal caso, a meno che ogni arco non sia bi-direzionale, verrebbe memorizzata letteralmente **la metà** dei nodi memorizzati in un grafo non orientato.

Osservazione 1.2.1

Supponiamo di avere un problema che ci richieda frequentemente quali nodi puntano verso un dato nodo u. Se mantenessimo un'implementazione mediante liste di adiacenza come quella vista in figura, per trovare tutti i nodi collegati ad u dovremmo scorrere per ogni nodo nella lista le sue liste di adiacenza, ottenendo un costo computazionale di $\mathcal{O}(n+m)$, come se fosse una normale visita.

Se invece decidessimo di modificare leggermente la struttura della lista di adiacenza,

aggiungendo per ogni nodo un puntatore ai suoi nodi entranti e uscenti, il costo computazionale di tale operazione si ridurrebbe ad un misero $\mathcal{O}(\deg(u))$, in quando dovremo scorrere solo la lista dei nodi entranti (il cui numero corrisponde appunto al grado del nodo). Nota bene che questa implementazione non appesantisce così tanto il costo di memorizzazione della struttura dati, in quanto per ogni nodo u si aggiunge $\mathcal{O}(\deg(u))$ di costo computazionale, portando il totale ad $\mathcal{O}(n + 3m = \mathcal{O}(n + m))$.

1.3 Passeggiate e cammini

Definizione 1.3.1: Cammino

Un **cammino** nel grafo è una passeggiata in cui non viene ripetuto alcun vertice.

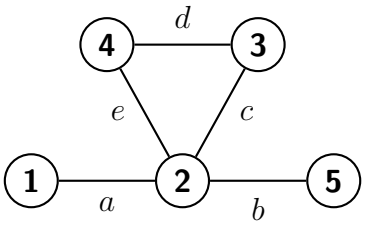


Figura 1.4: Una passeggiata potrebbe essere: 1 a 2 c 3 d 4 e 2 b 5.
Una camminata da 1 a 5 potrebbe invece essere 1 a 2 b 5.

Corollario 1.3.1

Esiste una passeggiata da x a y $\iff$ esiste un cammino da x a y.

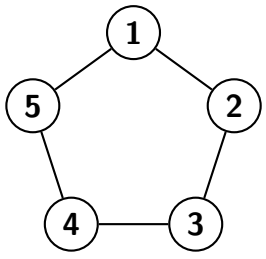
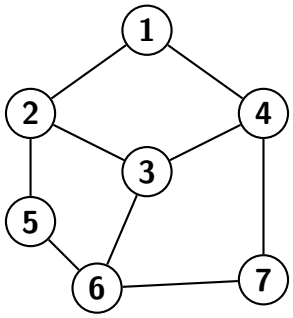


Figura 1.5: Grafo ciclo a 5 vertici (C_5).

1.4 Algoritmo di ricerca di un ciclo all'interno di un grafo

Proponiamo un altro problema classico del lavoro con i grafi: supponiamo che, dato un grafo G con l'ipotesi che **ogni vertice ha grado 2 o superiore**, volessimo trovare un

qualsiasi ciclo nel grafo. Partiamo con un esempio grafico di un grafo che contiene un ciclo, e vediamo come procederemmo umanamente.



L'idea di algoritmo chiaramente consiste nel:

1. Scegliere un nodo qualsiasi di partenza.
2. Memorizzarlo all'interno di un apposita struttura dati.
3. Prendere un arco e andare al nodo adiacente. **Importante** è assicurarsi che il prossimo nodo non sia ancora stato visitato. In tal caso, significa che abbiamo fatto un giro e abbiamo trovato un ciclo.
4. Altrimenti, tornare all'inizio.

Messa un'idea di base, procediamo con la scrittura dell'effettivo algoritmo in pseudocodice e in seguito alla sua spiegazione.

```
1     def trovaCiclo(G):
2       L = lista vuota
3       x = un vertice qualsiasi del grafo
4       L.append(x)
5
6       current = x
7       next = x.neigbh[0]
8
9       while next not in L:
10        L.append(next)
11
12        if next.neighbr[0] == current:
13          u = next.neighbr[1]
14        else:
15          u = next.neighbr[0]
```

```
16
17          current = next
18          next = u
19
20        while L[0] != next:
21          L.remove(0)
22
23        return L
```

Listing 1.1: Pseudocidice per la ricerca di un ciclo all'interno di un grafo.

Spiegazione algoritmo 1.4.1: Eestrazione di un ciclo da un grafo

1. Definiamo una lista vuota, per tenere traccia dei nodi visitati finora.
2. Prendiamo un qualsiasi vertice dal quale iniziare la nostra ricerca.
3. Aggiungiamo il nodo scelto all'interno della lista.
4. Prendiamo come prossimo nodo il primo della lista degli adiacenti al nodo corrente.
5. Eseguiamo un ciclo che va avanti finché il next che abbiamo preso non è un nodo già visitato: in tal caso significa che abbiamo intrapreso una strada circolare e siamo tornati indietro.
6. All'interno del ciclo aggiungiamo alla lista `L` dei nodi visitati anche `next`.
7. A questo punto dobbiamo scegliere il nuovo `next`, ma dobbiamo assicurarci che non sia il nodo da cui siamo partiti: ecco perché controlliamo che `next.neighbr[0]` sia diverso dal nodo su cui siamo adesso, in modo da non poter tornare *subito* indietro.
8. Aggiorniamo i puntatori dei nodi per la prossima iterazione.
9. Una volta usciti da questo ciclo, significa che abbiamo intrapreso la strada circolare di cui si parlava prima, e che dunque abbiamo trovato un ciclo. Significa inoltre che ora il puntatore `next` punterà al nodo da cui inizia la diramazione del ciclo, e dunque sarà il primo di questo. Ecco perché, nel ciclo finale, controlliamo che `L[0] != next`, perché `next` è il primo nodo del ciclo, e dunque deve essere anche il primo della lista che dovremo restituire.

Bisogna fare un appunto su questo algoritmo: ora come ora, per come l'abbiamo scritto, il suo costo computazionale è un $\mathcal{O}(n^2)$, tutto per colpa del controllo di testa del primo ciclo while. La condizione del ciclo è un `next not in L`, che richiede tempo $\mathcal{O}(n)$ in un array disordinato. Dato che il ciclo while potrebbe poi richiedere fino ad un massimo di n iterazioni, una per ogni nodo del grafo, il costo computazionale sale al tempo quadratico. Possiamo ovviamente ottimizzare questo codice, presentando la seguente versione:

```
    def trovaCiclo(G):
      L = lista vuota
      vis = [0] * n

      x = un vertice qualsiasi del grafo
      L.append(x)
      vis[x] = 1

      current = x
      next = x.neigbh[0]

      while vis[next] == 0:
        L.append(next)
        vis[next] = 1

        if next.neighbr[0] == current:
          u = next.neighbr[1]
        else:
          u = next.neighbr[0]

        current = next
        next = u

      while L[0] != next:
        L.remove(0)

    return L
```

Listing 1.2: Ricerca di un ciclo in un grafo ottimizzata

Spiegazione algoritmo 1.4.2: Ricerca di un ciclo in un grafo

Andremo a commentare solo le nuove scelte implementative: Si è scelto di aggiungere un array `vis` che verrà usato per tenere traccia degli elementi visitati. Ciò viene fatto stabilendo una biezione tra l'identificativo del nodo e il suo indice all'interno dell'array `vis`. Così facendo il controllo in testa al ciclo while diventa semplicemente un controllo accedendo ad un elemento dell'array tramite indice, dunque eseguibile in tempo costante. L'aggiunta di un array `vis` che ha una grandezza massima dell'ordine di del numero di nodi n aumenta il costo spaziale da $\Theta(n)$ a $\Theta(n+n) = \Theta(n)$, dunque rimane nello stesso ordine asintotico.

Possiamo ora discutere riguardo al costo computazionale dell'algoritmo, per entrambe le versioni:

- Versione inefficiente:
  - **Costo computazionale:** $\mathcal{O}(n^2)$
  - **Costo spaziale:** $\Theta(n)$
- Versione efficiente:
  - **Costo computazionale:** $\Theta(n)$
  - **Costo spaziale:** $\Theta(n)$

1.5 DFS - Depth-First-Search

Prima di discutere i modi in cui si possono effettuare le visite su un grafo, diamo alcune definizioni importanti.

Definizione 1.5.1: Nodo raggiungibile

Un nodo y si dice **raggiungibile** da x se $\exists$ un cammino da x a y.

Definizione 1.5.2: Grafo connesso

Un grafo G si dice **connesso** se $\forall x, y \in V(G)$, $\exists$ un cammino da x a y.

Le visite nascono per rispondere alla problematica classica: *Esiste un cammino da x a y?*

Il metodo più sicuro consiste nel **visitare**, a partire dal nodo x, uno ad uno tutti i nodi del grafo. Ovviamente, se durante tale visita si incontra il nodo y, la risposta alla domanda sarà affermativa.

La **visita in profondità**, o **Depth-First-Search (DFS)**, funziona come l'esplorazione di un labirinto: si sceglie un percorso e si avanza finché non si trova un vicolo cieco. A quel punto si torna indietro fino al primo incrocio in cui si sono lasciate strade inesplorate e si prova un'altra via.

In base alla brevissima e introduttiva descrizione dell'algoritmo fornita poco fa, possiamo delinearne un'idea algoritmica di base:

1. Partiamo da un nodo x e lo segniamo come visitato.
2. Scegliamo una strada inesplorata verso cui procedere.
3. Segniamo il nodo scelto come visitato e ripetiamo la visita ricorsivamente o iterativamente.

La modalità di backtracking verrà implementata mediante una pila (*stack*), che lavora secondo il principio LIFO (*Last In, First Out*). Dunque, quando dobbiamo tornare indietro, basta effettuare un'operazione di `pop()` per tornare al nodo precedentemente esaminato.

Vediamo una prima implementazione iterativa dell'algoritmo:

```
DFS(G, x):
  vis = [0] * n
  S = uno stack
  S.push(x)
  vis[x] = 1

  while S.size() != 0:
    y = S.top()
    if esiste un vicino z di y che non è stato visitato (
       vis[z] == 0):
      S.push(z)
      vis[z] = 1
    else:
      S.pop()

  return vis
```

Listing 1.3: Implementazione iterativa della DFS

Spiegazione algoritmo 1.5.1: Analisi dettagliata dell'algoritmo DFS

Analizziamo riga per riga il funzionamento dell'implementazione:

- `vis = [0] * n`: Inizializziamo l'array per tenere traccia dei nodi visitati.
- `S = uno stack`: Inizializziamo lo stack per permettere il backtracking.
- `S.push(x)` e `vis[x] = 1`: Aggiungiamo il nodo di partenza allo stack e lo segniamo come visto.
- `while S.size() != 0`: Iteriamo finché lo stack non rimane vuoto, il che vorrà dire che ogni nodo raggiungibile è stato esaminato.
- `y = S.top()`: Prendiamo il nodo in cima allo stack, che rappresenta il nodo corrente.
- **Controllo del vicino**: Se la condizione di entrata è rispettata (esiste un vicino z tale che `vis[z] == 0`), vuol dire che possiamo continuare per un'altra strada, cioè quella verso z. Di conseguenza effettuiamo il `push` di z e lo marchiamo come visitato.
- **Gestione del vicolo cieco (*else*)**: Altrimenti, significa che abbiamo raggiunto un "vicolo cieco" e che dobbiamo tornare indietro. Nel pratico, questo si traduce nel rimuovere l'elemento in cima alla pila tramite `S.pop()`, così che alla prossima iterazione i controlli vengano fatti sul nodo da cui siamo arrivati e sui suoi rimanenti vicini.

Seppur si tratti di un'implementazione molto descrittiva dell'algoritmo, analizziamo il controllo condizionale: dobbiamo prendere il nodo y, controllarne i vicini e verificare che almeno per un vicino z valga la condizione `vis[z] == 0`.

Questo controllo corrisponde a una ricerca sequenziale che, come ben sappiamo, richiede tempo $O(n)$ nel caso peggiore per singolo nodo, se non opportunamente ottimizzata.

È poi facile verificare come il ciclo `while` si ripeta tante volte quanti sono i nodi del grafo, dunque $\Theta(n)$.

Combinando queste due osservazioni, otteniamo che il costo di questa versione è $O(n^2)$, che risulta non essere particolarmente ottimale.

Proponiamo di seguito una versione ottimizzata dell'algoritmo.

```
1     def DFS_ottimizzato(G, x):
2       vis = [0] * len(G)
3       vis[x] = 1
4
5       s = Stack()
6       s.push(x)
7
8       while s.size() != 0:
9         current = s.pop()
10        for v in current.neighbor: # (2)
11          if vis[v] == 0:
12            vis[v] = 1
13            s.push(v)
14
15      return vis
```

Spiegazione algoritmo 1.5.2: Visita DFS ottimizzata

La differenza rispetto alla versione non ottimizzata risiede in due passaggi chiave:

1. In **(1)** estraiamo il nodo corrente dalla pila. Così facendo lo eliminiamo dalla pila e non dal grafo, evitando modifiche distruttive alla struttura dati; in questo modo il nodo `current` non verrà mai più controllato nelle iterazioni successive.

2. In **(2)** andiamo a verificare ciascun vertice adiacente a quello appena estratto. Se non è ancora stato visitato, lo marchiamo come visitato e lo inseriamo nella pila, così da poter proseguire l'esplorazione a partire da esso.

Questa versione dell'algoritmo presenta la seguente complessità:

- **Costo spaziale**: $\Theta(n)$, dovuto all'array dei nodi visitati e allo spazio occupato dalla pila nel caso peggiore.
- **Costo temporale**: il ciclo `while` viene eseguito n volte, esattamente una volta per ciascun vertice. All'interno del ciclo viene eseguito un lavoro costante $\Theta(1)$ per l'operazione di estrazione (`pop`) e un costo pari a $O(\deg(\text{current}))$ per scorrere l'intera lista di adiacenza del nodo estratto.

Il costo computazionale complessivo del ciclo è quindi descritto dalla seguente sommatoria:

$$\sum_{i=1}^{n}\Big(O(1)+O(\deg(G[i]))\Big)=\sum_{i=1}^{n}1+\sum_{i=1}^{n}\deg(G[i])=$$
$$=O(n)+O(2m)=O(n+2m)=O(n+m)$$

Per completezza, essendo i grafi un soprainsieme degli alberi, scriviamo una versione ricorsiva della DFS divisa in una funzione ausiliaria e nella visita effettiva.

```
def dfs(G, x):
  vis = [0] * n
  return dfs_ric(G, x, vis)

def dfs_ric(G, x, vis):
  vis[x] = 1
  for v in x.neighbr:
    if vis[v] == 0:
      dfs_ric(G, v, vis)
```

La visita costa sempre $O(n+m)$. L'unica limitazione è lo stack delle funzioni, che per grafi molto pieni potrebbe andare in *recursion depth*.

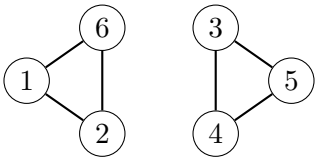
Proponiamo ora un esercizio che ci aiuterà a lavorare con i grafi e con la visita in profondità (DFS).

Esercizio 1.5.1: Calcolo delle componenti connesse tramite DFS

Modificare l'algoritmo di visita DFS al fine di calcolare num_comp, ovvero il numero totale di componenti connesse di un grafo, e popolare un array comp[] con valori appartenenti a $\{1,\dots,\text{num_comp}\}$ tale che:

$$\text{comp}[x]=\text{comp}[y] \iff x \text{ e } y \text{ appartengono alla stessa componente connessa}$$

Esempio. Consideriamo il seguente grafo disconnesso formato da due componenti connesse:



In questo caso il risultato dell'algoritmo sarà:

$$\text{num_comp} = 2, \qquad \text{comp} = [1, 1, 2, 2, 2, 1]$$

dove la posizione i-esima dell'array comp indica la componente di appartenenza del nodo i.

Diamo prima un'idea generale della struttura dell'algoritmo:

1. **Inizializzazione delle strutture dati globali**: Inizializziamo l'array globale comp di dimensione n impostando tutti i valori a 0. Impostiamo inoltre la variabile globale $\text{num_comp} = 0$.

2. **Avvio della DFS**: Selezioniamo il primo nodo non ancora visitato, incrementiamo il contatore ($\text{num_comp} = \text{num_comp} + 1$) ed eseguiamo la DFS a partire da esso.

3. **Assegnazione della componente durante la DFS**: All'interno della funzione DFS, per ogni vertice v raggiunto durante la visita, impostiamo:

   $$\text{comp}[v] = \text{num_comp}$$

   L'utilità dell'array `comp` sta anche nel fatto che può benissimo rimpiazzare l'array `vis` che avevamo precedentemente usato per marcare i nodi visitati. Avendolo inizializzato con tutti 0, i nodi non ancora visitati avranno `comp[v] = 0`.

4. **Scorrimento di tutti i vertici**: Terminata la singola DFS, si prosegue scorrendo l'elenco dei vertici del grafo:

   - Se `comp[v] != 0`, il nodo è già stato visitato e possiamo passare al successivo.
   - Altrimenti, incrementiamo `num_comp = num_comp +1` e riavviamo la DFS dal nodo v.

Avendo discusso l'idea dell'algoritmo, possiamo fornirne un esempio di implementazione

```
1     def trova_comp(G):
2       comp = [0] * n
3       num_comp = 0
4       S = Stack()
5
6       for v in G:
7         if comp[v] == 0:
8           num_comp += 1
```

```
            S.push(v)
            comp[v] = num_comp

            while S.size() != 0:
              current = S.pop()
              for w in current.neighbor:
                if comp[w] == 0:
                  comp[w] = num_comp
                  S.push(w)

        return comp, num_comp
```

Spiegazione algoritmo 1.5.3: Ricerca delle componenti connesse

- **Inizializzazione**: Viene creato un vettore `comp` di dimensione n inizializzato a 0, che servirà anche a segnalare i nodi visitati, un contatore `num_comp` impostato a 0 e una struttura dati di tipo `Stack`.
- **Identificazione di una nuova componente**: Nel ciclo `for v in G`, l'istruzione `if comp[v] == 0` verifica se il nodo v non è ancora stato visitato. Se questa condizione è vera, significa che ci troviamo su una **nuova componente connessa**, in quanto il nodo corrente non appartiene a nessuna componente precedentemente esplorata.
- **Assegnazione ed esplorazione**:
  1. Si incrementa il contatore delle componenti (`num_comp += 1`).
  2. Si inserisce il nodo sorgente v nello stack `S` e se ne assegna l'indice della componente corrispondente (`comp[v] = num_comp`).
  3. Tramite il ciclo `while S.size() != 0`, si procede all'estrazione di ogni nodo (`current = S.pop()`) ed all'analisi dei suoi vicini (`for w in current.neighbor`). Per ciascun vicino non ancora visitato (`comp[w] == 0`), viene registrata l'appartenenza alla componente corrente e il nodo viene inserito nello stack per ulteriori esplorazioni.

Il costo computazionale dell'algoritmo è lo stesso di una normale DFS, dunque risulta essere $\mathcal{O}(n+m)$. Si potrebbe pensare che la DFS venga eseguita una volta per ogni nodo, ma per come abbiamo segnalato i nodi visitati, l'esplorazione viene eseguita **una sola**

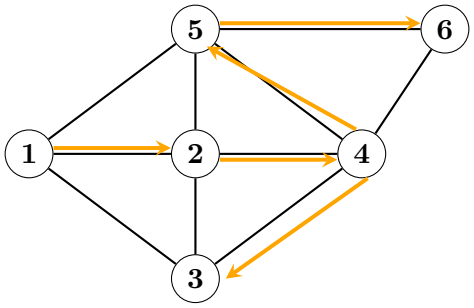
volta per ogni componente connessa.

1.6 Generici problemi su grafi

Prima di continuare il discorso sulle visite, vediamo alcuni esempi di operazioni sui grafi diretti (o orientati).

Definizione 1.6.1: Albero di ricerca

Dati due nodi x e y, gli archi che utilizziamo per raggiungere y a partire da x formano il cosiddetto **albero di ricerca**.
Si ricorda che un albero è definito come un **grafo connesso e aciclico**.



Tale albero viene memorizzato mediante il cosiddetto **"vettore dei padri"**, ovvero un array in cui, per ogni indice i, memorizziamo la provenienza dell'esplorazione.

Nell'esempio precedente, il vettore dei padri è rappresentato da:

$$P = \begin{bmatrix} 1 & 2 & 3 & 4 & 5 & 6 \\ 1, & 1, & 4, & 2, & 4, & 5 \end{bmatrix}$$

in cui, per ogni elemento ad indice i, è specificato da quale nodo proviene i.

Il nodo radice dell'albero si trova identificando quell'elemento in cui $P[i] == i$, cioè il valore del nodo è uguale al suo indice nell'array.

Esercizio 1.6.1: Distanza di un nodo dalla radice

Dato un vettore dei padri P e un vertice x, vogliamo trovare la distanza di x dalla radice dell'albero.

```
1    def distRad(P, x):
2      count = 0
3      while P[x] != x:
4        x = P[x]
```

```
5           count += 1
6         return count
```

Listing 1.4: Risoluzione iterativa

```
1     def distRad(P, x):
2       if P[x] == x:
3         return 0
4       return 1 + distRad(P, P[x])
```

Listing 1.5: Risoluzione ricorsiva

Questo algoritmo potrà essere utilizzato in futuro nel corso.

Avendo definito il vettore dei padri, possiamo implementare la visita in profondità (DFS) sfruttando tale struttura.

```
1     def dfs_ric(G, x, P):
2       for v in x.neighbor:
3         if P[v] == null:
4           P[v] = x
5           dfs_ric(G, v, P)
6
7     def dfs(G, x):
8       P = [null] * n
9       P[x] = x
10      dfs_ric(G, x, P)
11      return P
```

Listing 1.6: Algoritmo DFS con vettore dei padri

1.6.1 Problema di bicolorazione di un grafo

Consideriamo un insieme di n corsi e m studenti. Vogliamo programmare gli esami in due settimane, cioè in modo tale che due esami differenti non possano essere collocati nella stessa settimana.

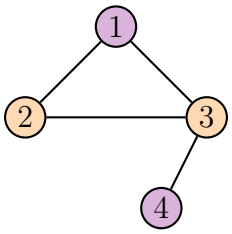
Supponiamo che ciascuno studente debba sostenere esattamente due corsi. Questa assunzione è motivata dal fatto che, se uno studente avesse un numero di corsi superiore a tre, sarebbe impossibile evitare di posizionarne almeno due nella stessa settimana.

E' molto importante che, nella risoluzione di un problema di grafi, sappiamo individuare quale dato nel problema corrisponde ai nodi e quale agli archi.

Nel nostro caso, consideriamo un esempio con 4 corsi e 4 studenti, dove ogni studente è associato a una coppia di corsi da frequentare e superare:

- Corso 1: studenti $\{1, 2\}$
- Corso 2: studenti $\{2, 3\}$
- Corso 3: studenti $\{3, 4\}$
- Corso 4: studenti $\{1, 3\}$

Come possiamo osservare, siccome ogni studente è una coppia di esami, questa corrisponde esattamente alla definizione di arco in un grafo. Dunque il grafo $G = (\text{Corsi}, \text{studenti})$ risultante per questo esempio è rappresentato di seguito:

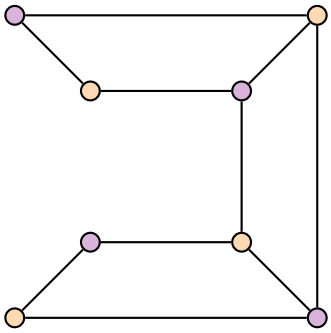


Dove i colori dei nodi indicano la settimana assegnata all'esame:

- Settimana 1
- Settimana 2

Come possiamo notare, nell'esempio fornito non è possibile soddisfare il requisito desiderato senza violare i vincoli, poiché i vertici 1, 2 e 3 formano un triangolo (un ciclo di lunghezza dispari).

Il problema della programmazione degli esami in due settimane si riconduce al classico **problema di bi-colorazione dei grafi**, ovvero determinare se è possibile assegnare uno di due colori (le due settimane) a ciascun vertice in modo tale che ogni arco congiunga due vertici di colore diverso.



Proposizione 1.6.1: Impossibilità della bi-colorazione

Il problema di bi-colorazione **non ha soluzione** per i grafi che contengono **cicli di lunghezza dispari**.

L'intuizione dietro questa proprietà è che, percorrendo un ciclo di lunghezza dispari alternando i colori, dovendo assegnare colori diversi a vertici adiacenti, si arriverà inevitabilmente a un punto in cui due vertici adiacenti richiederanno lo stesso colore, rendendo impossibile una colorazione valida.

Quando poniamo un nodo ad un gruppo (ad esempio 1), allora tutti i suoi adiacenti dovranno appartenere all'altro gruppo (2) e viceversa.

Vediamo l'effettivo codice che, preso in input un grafo, restituisce un Array `part` rappresentante l'eventuale partizione.

```
    def dfs_part(G, x):
      part = [0] * n
      part[x] = 1
      s = stack()
      s.push(x)

      while s.size() != 0:
        y = s.top()
        for v in y.neighbr:
          if part[v] == 0:
            part[v] = 3 - part[y]
            s.push(v)
          else:
            if part[v] == part[y]:
              return []
      return part
```

Spiegazione algoritmo 1.6.1: Spiegazione dell'algoritmo di partizione

Il codice sfrutta una visita in profondità (DFS) modificata per verificare se un grafo ammette una partizione (ad esempio se è bipartito).

- L'array `part` tiene traccia dello stato di colorazione di ciascun nodo: 0 indica non visitato/non colorato, mentre 1 e 2 rappresentano le due classi di partizione.
- Per ogni adiacente v del nodo corrente y:
  1. Se v non è stato ancora visitato ('part[v] == 0'), gli viene assegnato il colore opposto rispetto a y tramite la formula $3-\text{part}[y]$, e viene aggiunto allo stack per la visita.
  2. Se v è già stato visitato, si controlla che non abbia lo stesso colore del nodo corrente y. In caso di conflitto (`part[v] == part[y]`), il grafo non può essere partizionato e l'algoritmo termina restituendo una partizione vuota.

1.7 Lavoro su grafi orientati

In un grafo orientato, la visita DFS funziona ugualmente, ma bisogna prestare attenzione all'orientamento degli archi. Tuttavia nel pratico non cambia nulla, in quanto nella lista di adiacenza di un nodo saranno presenti solo i riferimenti ai nodi che può effettivamente raggiungere.

Inoltre, l'**albero delle visite generato dalla DFS su un grafo diretto** si dice **arborescenza**.

Esercizio 1.7.1: Ordinamento topologico di un grafo

Si consideri un processo suddiviso in n sotto-attività (o *task*) $x_1, \ldots, x_n$, soggette a una serie di vincoli di precedenza:

- x_1 prima di x_2
- x_3 prima di x_2
- x_3 prima di x_1
- x_4 prima di x_3

Trovare un ordine di task che rispetti le varie dipendenze fornite in input.

È naturale modellare le attività x_i come i nodi di un grafo diretto. Per quanto riguarda gli archi, stabiliamo la seguente convenzione grafica per rappresentare la relazione di dipendenza temporale:

$$\text{“}x_i \text{ viene prima di } x_j\text{”} \quad \Longleftrightarrow \quad x_i \longleftarrow x_j$$

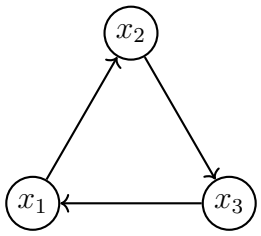
Definizione 1.7.1: Ordine topologico

Risolvere questo problema equivale a individuare un **ordine topologico** dei nodi del grafo, ovvero una permutazione dei suoi vertici $(v_1, v_2, \ldots, v_n)$ tale che, per ogni arco orientato $(u, v) \in E$, il vertice u preceda il vertice v nell'ordinamento.

Chiaramente, una soluzione non esiste sempre. Si consideri ad esempio il seguente insieme di vincoli:

- x_1 prima di x_3
- x_3 prima di x_2
- x_2 prima di x_1

Rappresentando graficamente queste dipendenze si ottiene una struttura ciclica:



Proposizione 1.7.1: Inesistenza dell'ordine topologico

Se il **grafo orientato** G definito dai processi **contiene un ciclo diretto**, allora **non esiste** alcun ordine topologico.

Proposizione 1.7.2: Proprietà dei cicli nei grafi diretti

Se $\nexists$ un ciclo in un grafo diretto $\implies \exists$ un ordine topologico.

Proposizione 1.7.3: Esistenza di vertici senza archi uscenti

Se G è un grafo diretto senza cicli $\implies \exists$ un vertice v senza archi uscenti.

Proposizione 1.7.4: Presenza di cicli nei grafi diretti

Per ogni vertice v, se $\exists$ almeno un arco uscente da $v \implies G$ contiene un ciclo.

Definizione 1.7.2: DAG (Directed Acyclic Graph)

Per notazione, indichiamo con **DAG** (*Directed Acyclic Graph*) un grafo diretto privo di cicli.

1.7.1 Ricerca di un ordine topologico

Vogliamo definire un algoritmo per trovare un **ordine topologico** in un grafo orientato aciclico (DAG).

Osservazione 1.7.1: Proprietà fondamentale

Per la proposizione precedente, sappiamo che se in un grafo orientato esiste un ordine topologico, allora esiste **almeno un nodo senza archi uscenti** (ovvero un nodo pozzo).

Da questa osservazione nasce l'idea per un primo algoritmo. L'idea di base consiste nell'eseguire iterativamente le seguenti operazioni:

1. Scorriamo tutti i vertici del grafo e individuiamo un vertice privo di archi uscenti.
2. Lo rimuoviamo dal grafo.
3. Ripetiamo il procedimento finché non sono stati rimossi tutti i vertici.

Formalizziamo questo approccio tramite il seguente pseudo-codice:

```
Ord(G: grafo diretto aciclico):
  L = []
  while len(G) != 0:
    v = trova_nodo_senza_archi_uscenti(G)
    L.append(v)
    cancella_nodo_e_archi(G, v)
  return L
```

Listing 1.7: Algoritmo ingenuo per l'ordine topologico

Spiegazione algoritmo 1.7.1: Ordine topologico (inefficiente)

Analizziamo nel dettaglio le singole istruzioni dell'algoritmo e i relativi costi computazionali:

- **Inizializzazione e ciclo principale**: Il ciclo `while` viene eseguito $\Theta(n)$ volte, poiché a ogni iterazione viene rimosso esattamente un nodo dal grafo.
- **Ricerca del nodo senza figli**: Per individuare un vertice senza archi uscenti (ovvero un nodo la cui lista di adiacenza sia vuota), dobbiamo scorrere i nodi del grafo, con un costo di $O(n)$.
- **Cancellazione del nodo**: La rimozione del nodo v dal grafo e la successiva eliminazione di tutte le sue occorrenze nelle liste di adiacenza degli altri nodi richiede di scandire l'intera struttura dati, con un costo pari a $O(n+m)$.

Il costo totale dell'algoritmo risulta quindi essere:

$$\Theta(n) \cdot O(n+m) = O(n(n+m))$$

Osservazione 1.7.2: Presenza di più nodi pozzo

Anche se nel grafo dovessero essere presenti più nodi contemporaneamente privi di archi uscenti, l'algoritmo funziona correttamente. Esso restituirà comunque un ordine topologico valido (ricordando che per uno stesso DAG possono esistere più ordini topologici distinti).

Come si nota dall'analisi della complessità, si tratta di un metodo **molto costoso** dal punto di vista computazionale, oltre al fatto che modifica in modo distruttivo la struttura del grafo originale.

Esercizio 1.7.2: Trovare un ciclo in un grafo diretto

Dato un grafo diretto G, trovare un ciclo al suo interno.

```
def trovaCicloDiretto(G):
    L = []  # Per memorizzare i nodi del ciclo
    x = vertice qualsiasi
    L.append(x)

    vis = [0] * n
    vis[x] = 1  # Segniamo il nodo come visitato

    current = x
    next = x.neighbr[0]

    # Iteriamo finche non troviamo un nodo gia visitato
    while vis[next] == 0:
        vis[next] = 1
        L.append(next)

        current = next
        next = current.neighbr[0]  # (1)

    while L[0] != next:
        L.remove()

    return L
```

Listing 1.8: Algoritmo per la ricerca di un ciclo diretto

Spiegazione algoritmo 1.7.2: Ricerca di un ciclo diretto

1. **Inizializzazione**: Viene creata una lista L per memorizzare il percorso e un vettore di tracciamento `vis` di dimensione n inizializzato a 0. Si seleziona un vertice di partenza arbitrario x, lo si inserisce in L e si imposta `vis[x] = 1`.

2. **Esplorazione**: Si imposta `current = x` e come vertice successivo `next` il primo adiacente uscente (`x.neighbr[0]`).

3. **Ricerca del ciclo**: Finché il nodo `next` non è stato visitato (`vis[next] == 0`):

   - Si marca `next` come visitato (`vis[next] = 1`).

- Si aggiunge `next` alla lista L.
- Si avanza ponendo `current = next` e selezionando il nuovo vicino uscente `next = current.neighbr[0]` (istruzione **(1)**).

4. **Estrazione della sequenza del ciclo**: Quando il ciclo `while` termina, significa che `next` è un nodo già visitato (è stato ri-incontrato). Il secondo ciclo `while` rimuove gli elementi in testa a L finché l'elemento di testa non coincide con `next`, lasciando in L solo ed esclusivamente i nodi che compongono il ciclo diretto trovato.

L'unica sostanziale differenza rispetto al caso non orientato è un'accortezza: nell'istruzione **(1)** possiamo permetterci di assegnare come prossimo nodo sempre il primo dei vicini (`current.neighbr[0]`). Infatti, essendo il grafo **diretto**, anche la presenza di un eventuale doppio arco orientato tra due soli nodi costituisce a tutti gli effetti un ciclo valido.

Esercizio 1.7.3: Problema dell'antenato comune più vicino

Dato P il vettore dei padri di un albero e dati due vertici x ed y, l'obiettivo è trovare l'antenato comune più vicino (noto anche come Lowest Common Ancestor o LCA) tra x e y.

Presentiamo una soluzione basata sull'utilizzo di liste puntate per risalire la gerarchia dei padri fino alla radice.

```
1   def sol1(P, x, y):
2     L1, L2 liste puntate
3     Do:
4       L1.push(x)
5       x = P[x]
6     while x != P[x]
7
8     Do:
9       L2.push(y)
10      y = P[y]
11    while y != P[y]
12
13    Current = L1[0]
14    while L1[1] == L2[1]:
```

```
15          Current = L1[1]
16          L1.remove()
17          L2.remove()
18
19      return Current
```

Osservazione 1.7.3

Essendo `L1` ed `L2` liste puntate, gli elementi vengono aggiunti in testa durante la risalita verso la radice. Di conseguenza, alla fine dell'inserimento, il primo elemento della lista corrisponde proprio alla radice dell'albero, mentre gli ultimi elementi rappresentano i nodi di partenza x ed y.

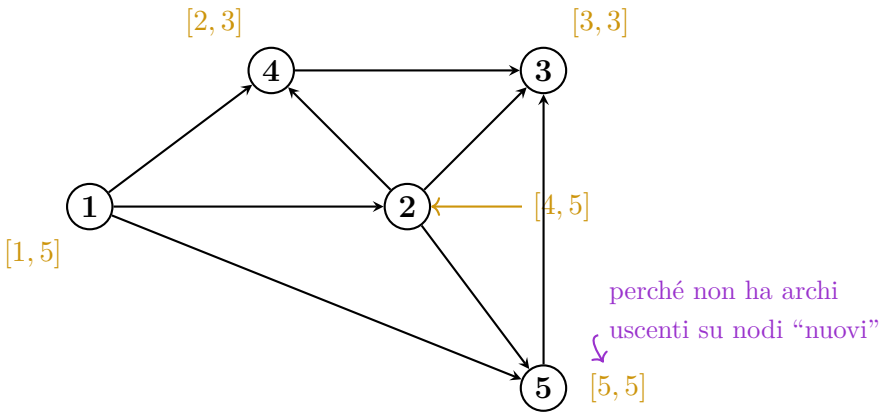
Tornando agli ordini topologici... In precedenza era stata presentata una versione inefficiente con complessità temporale pari a $O(n(n+m))$. È possibile progettare una versione decisamente più efficiente sfruttando una visita in profondità (**DFS**).

Prima di definire formalmente l'algoritmo, introduciamo alcune nozioni teoriche fondamentali relative alla gestione del tempo durante la visita.

Definizione 1.7.3: Tempi di scoperta e fine visita in DFS

Dato un grafo G e una visita DFS su di esso, introduciamo un **contatore globale che viene incrementato ad ogni nuovo nodo visitato**. Ad ogni nodo v viene associata una coppia di valori temporali:

- **Tempo di scoperta** $t(v)$: indica il valore del contatore nel momento in cui il nodo v viene inserito nello stack della DFS. Sostanzialmente, rappresenta l'istante in cui il vertice viene visitato per la prima volta.
- **Tempo di fine visita** $T(v)$: indica il valore del contatore nel momento in cui il nodo v viene rimosso dallo stack della DFS. Sostanzialmente, indica quanti nodi sono stati esplorati complessivamente prima di eliminare v dallo stack (ovvero quando l'esplorazione di v e di tutti i suoi discendenti è terminata).



Consideriamo il grafo sopra rappresentato ed analizziamo l'evoluzione della visita in profondità (DFS):

1. **Partenza dal nodo radice**: supponiamo di partire dal nodo 1. Poniamo $t(1) = 1$, in quanto è il primo vertice visitato.
2. **Attraversamento di un arco uscente**: scegliamo un arco uscente arbitrario, ad esempio verso il nodo 4. Poniamo $t(4) = 2$, essendo il secondo vertice visitato.
3. **Avanzamento**: ci muoviamo verso l'unico nodo adiacente disponibile, il nodo 3, e poniamo $t(3) = 3$.
4. **Backtracking da nodo senza uscite non visitate**: poiché il nodo 3 non possiede archi uscenti verso nodi ancora da scoprire, lo rimuoviamo dallo stack per effettuare il backtracking. Poniamo quindi $T(3) = 3$. Si noti che si tratta dello stesso contatore, ma in questo passaggio non viene incrementato.
5. **Ritorno al predecessore**: torniamo al nodo 4; dato che non ha altri archi uscenti verso nodi non ancora visitati, lo eliminiamo dallo stack e assegniamo $T(4) = 3$.
6. **Prosecuzione**: si procede in modo analogo per i restanti vertici del grafo.

Da questo meccanismo possiamo ricavare due proprietà fondamentali sui tempi di scoperta e di fine visita:

Osservazione 1.7.4: Tempi della radice

Se il grafo è connesso e v è la radice dell'albero di visita, si ha necessariamente:

$$t(v) = 1, \qquad T(v) = n$$

Ciò è dovuto al fatto che la radice è il primo nodo ad essere scoperto ed è l'ultimo ad essere rimosso dallo stack.

Osservazione 1.7.5: Relazione tra tempo di inizio e fine visita

Per ogni vertice $v \in V$ vale la disuguaglianza:

$$t(v) \leq T(v)$$

in quanto un nodo deve essere necessariamente scoperto prima di poter terminare la sua elaborazione ed essere rimosso.

Dunque, ad ogni vertice v risulta associato un intervallo temporale ben definito:

$$[t(v), T(v)]$$

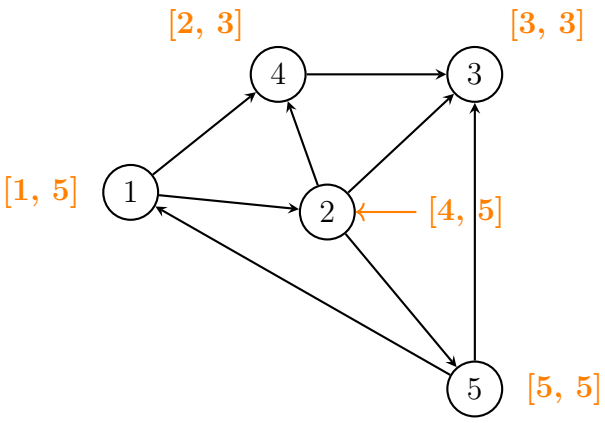
Proposizione 1.7.5: Classificazione degli archi in una DFS

Per ogni coppia di vertici u, v, con u la coda dell'arco e v la testa, vale solo una di queste relazioni:

i) $[t(u), T(u)] \subseteq [t(v), T(v)]$: **ARCO ALL'INDIETRO**

ii) $[t(v), T(v)] \subseteq [t(u), T(u)]$: **ARCO IN AVANTI**

iii) $[t(u), T(u)] \cap [t(v), T(v)] = \emptyset$: **ARCO DI ATTRAVERSAMENTO**



In questo esempio abbiamo:

- $(1, 4)$ = AVANTI
- $(4, 3)$ = AVANTI
- $(1, 2)$ = AVANTI
- $(2, 3)$ = ATTRAVERSAMENTO
- $(2, 5)$ = AVANTI
- $(5, 3)$ = ATTRAVERSAMENTO
- $(5, 1)$ = INDIETRO
- $(2, 4)$ = ATTRAVERSAMENTO

Ragionando tenendo conto dell'arborescenza, cioè dell'albero delle visite che si crea dalla successione dei nodi che appaiono in una DFS su un grafo diretto, vediamo come:

- **ARCHI IN AVANTI:** archi che vanno da un nodo ad un suo **discendente**.
- **ARCHI ALL'INDIETRO:** archi che vanno da un nodo ad un suo **antenato**.
- **ARCHI DI ATTRAVERSAMENTO:** archi che vanno da un nodo ad un suo **"cugino"**.

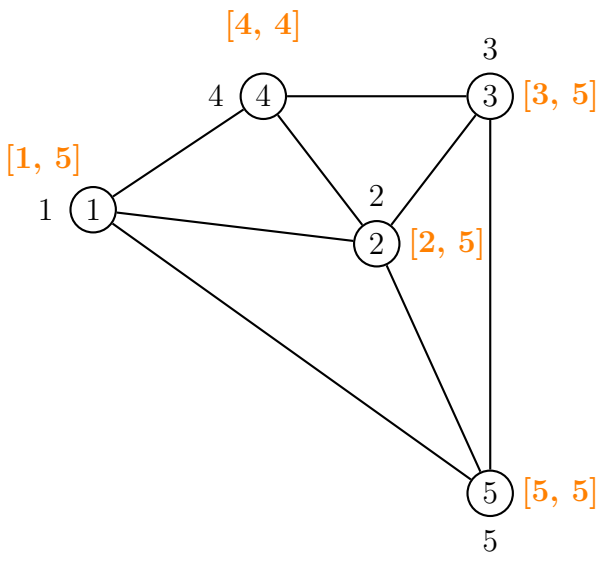
1.7.2 Comportamento nei grafi non diretti

Come si comporta questa implementazione nei **grafi non diretti**?

Siccome il grafo non è orientato, si possono verificare solo due possibili casi per gli intervalli di visita di qualsiasi coppia di nodi u, v:

i/ii) $I_u \subseteq I_v$ oppure $I_v \subseteq I_u$

iii) $I_u \cap I_v = \emptyset$



Nota: possiamo scegliere noi un ordine di visita dei nodi, di conseguenza l'intervallo segnato nel grafo è riportato solo a scopo esemplificativo.

Proposizione 1.7.6

Dato un DFS su un grafo **non diretto**, se un arco (u, v) non fa parte dell'**albero di visita** (ovvero è un arco all'indietro/back edge), allora è impossibile che si verifichi il caso **iii)** degli intervalli disgiunti.

Osservazione 1.7.6: Archi fuori dall'albero in grafi non orientati

Tutti gli archi (u, v) che non fanno parte dell'albero di visita di una DFS su un grafo non orientato sono necessariamente **archi all'indietro** (o in avanti; non essendoci un orientamento nel grafo, la distinzione non è rilevante).

Proposizione 1.7.7: Caratterizzazione dei cicli in grafi non orientati

Sia G un grafo non orientato connesso.

$$G \text{ ha un ciclo} \iff \text{ogni DFS ha almeno un arco all'indietro}$$

Proposizione 1.7.8: Caratterizzazione dei cicli in grafi orientati

Dato un grafo orientato G:

$$\exists \text{ un ciclo in } G \iff \exists \text{ un arco all'indietro per qualsiasi DFS nel grafo}$$

Torniamo molto a monte: vogliamo usare tutte queste supposizioni per dare un codice per trovare l'ordine topologico in un tempo $O(n + m)$ usando la DFS.

Programmiamo ora una DFS ricorsiva che ci permette di ottenere $t[x]$ e $T[x]$ per ogni vertice.

```
    def DFS_ric(G, x, t, T, c):
      for y in x.neighbor:
        if t[y] == -1:
          c += 1
          t[y] = c
          DFS_ric(G, y, t, T, c)
      T[x] = c

    def DFS(G, x):
      t = [-1] * n
```

```
        T = [-1] * n
        c = 1
        t[x] = c
        DFS_ric(G, x, t, T, c)
```

Spiegazione algoritmo 1.7.3: DFS con tempi di visita

La seguente procedura implementa la visita DFS tenendo traccia dei tempi di scoperta $t[x]$ e di fine visita $T[x]$ per ciascun vertice x.

Spiegazione dettagliata delle istruzioni:

- **Controllo della visita (`t[y] == -1`):** Usiamo l'array t per controllare se abbiamo già visitato il vertice y. Poiché $t[y]$ indica il tempo in cui abbiamo scoperto y, un valore pari a -1 indica che il nodo non è ancora stato visitato.
- **Fine visita del sottoalbero (`T[x] = c`):** Una volta che abbiamo visitato tutto il sottoalbero di x, dobbiamo rimuoverlo dallo stack e andare avanti, dunque valorizziamo il tempo di fine visita $T[x]$ con il valore corrente del contatore c (questo avviene in quanto non stiamo usando uno stack esplicito, bensì lo stack della ricorsione delle chiamate di funzione: la risalita dello stack della ricorsione corrisponde alla rimozione graduale dei nodi dallo stack).
- **Inizializzazione:** Nella funzione principale `DFS(G, x)` inizializziamo i due array t e T di dimensione n con il valore -1, impostiamo il contatore iniziale $c = 1$, marchiamo il nodo di partenza x e infine avviamo la procedura ricorsiva.

Per l'ordine topologico sappiamo che non ci sono archi all'indietro, essendo il grafo aciclico (DAG).

Di conseguenza, per qualsiasi arco (u, v), vale esclusivamente una delle seguenti relazioni tra gli intervalli temporali di visita:

- **Disgiunzione:**
$$[t[v], T[v]] \cap [t[u], T[u]] = \emptyset$$
- **Inclusione:**
$$[t[v], T[v]] \subseteq [t[u], T[u]]$$
In questo caso, l'arco (u, v) rappresenta un arco in avanti oppure un arco di **arborescenza**.

Dunque, per ogni arco (u, v) del grafo, vale sempre la relazione:

$$T[v] \leq T[u]$$

Per ricostruire un ordinamento topologico, dobbiamo considerare l'array dei tempi di fine visita T in ordine decrescente: infatti, l'ultimo vertice ad essere completato nella visita è il primo nell'ordine topologico.



Con un arco che collega x e y, si ha $T[y] \leq T[x]$ e quindi l'arco è orientato come $x \to y$.

Se invece consideriamo i tempi di fine visita T in ordine crescente, il primo vertice ad essere rimosso risulterà essere l'ultimo nell'ordinamento topologico.



Con un arco che collega x e y, si ha $T[y] \geq T[x]$ e quindi l'arco è orientato come $y \to x$.

Abbiamo dunque tutti gli elementi teorici necessari per definire l'algoritmo di ordinamento topologico basato su DFS.

```
def OrdTop(G):
  L = []
  vis = [0] * n
  for v in G:
    if vis[v] == 0:
      OrdTopRic(G, v, vis, L)
  return L

def OrdTopRic(G, v, vis, L):
  vis[v] = 1
  for u in v.neighbors:
    if vis[u] == 0:
      OrdTopRic(G, u, vis, L)
  L.append(v)
```

Spiegazione algoritmo 1.7.4: Ordinamento Topologico

- **Inizializzazione (`OrdTop`):** viene predisposta una lista L inizialmente vuota per raccogliere i nodi ordinati e un vettore booleano `vis` di dimensione n inizializzato a 0, impiegato per tracciare lo stato di visita di ciascun vertice.
- **Ciclo principale:** per ogni vertice $v \in G$, se v non è ancora stato visitato (`vis[v] == 0`), si avvia la procedura ricorsiva `OrdTopRic(G, v, vis, L)`. Al termine delle chiamate per tutte le componenti connesse, viene restituita la lista L.
- **Procedura ricorsiva (`OrdTopRic`):**
  1. Il vertice corrente v viene marcato come visitato impostando `vis[v] = 1`.
  2. Si esaminano tutti i vertici u uscenti da v (`v.neighbors`): se un vicino u non è ancora stato visitato (`vis[u] == 0`), la funzione richiama ricorsivamente se stessa su u.
  3. Non appena tutti i discendenti di v sono stati esplorati (visita in post-ordine), il nodo v viene inserito in L tramite l'operazione `L.append(v)`. L'inserimento del vertice v come ultimo elemento della lista è un modo molto furbo di utilizzare la risalita dello stack della ricorsione per inserire per ultimo letteralmente *l'ultimo vertice a cui siamo arrivati*, ovvero quello con la priorità minore (l'ultimo dell'ordine topologico).

La procedura richiama perfettamente una classica operazione di DFS, dunque il costo computazionale sarà $\mathbf{O(n + m)}$.

Definizione 1.7.4: Pozzo universale

In un grafo diretto G, un **pozzo universale** è un vertice $x \in V(G)$ tale che:

$$\forall y \in V(G) \setminus \{x\}, \quad (y, x) \in E(G) \land (x, y) \notin E(G)$$

In termini intuitivi, si tratta di un nodo a cui arrivano archi da tutti gli altri vertici del grafo, ma dal quale non parte alcun arco verso l'esterno.

Esercizio 1.7.4: Ricerca del pozzo universale in tempo

Dato un grafo G con n vertici memorizzato tramite una matrice di adiacenza, l'obiettivo è stabilire se esiste un pozzo universale in tempo lineare $O(n)$.

Idea intuitiva La matrice di adiacenza ci permette di verificare la presenza di un arco tra due nodi i e j in tempo costante $O(1)$. Da un singolo controllo sulla cella $G[i][j]$ possiamo ricavare due informazioni cruciali che ci consentono di escludere uno dei due nodi come potenziale pozzo:

- **Se** $G[i][j] == 0$: significa che non esiste un arco orientato da i a j. Di conseguenza, j **non può** essere un pozzo universale, poiché un pozzo universale deve essere raggiunto da tutti gli altri nodi del grafo (incluso i).
- **Se** $G[i][j] == 1$: significa che esiste un arco orientato da i a j. Di conseguenza, i **non può** essere un pozzo universale, poiché da un pozzo universale non deve partire alcun arco verso l'esterno.

Grazie a questa proprietà, con un singolo confronto siamo in grado di scartare un candidato, riducendo progressivamente lo spazio di ricerca.
Inoltre, per la definizione di pozzo universale, possiamo ricavare un'altra informazione dalla matrice di adiacenza.

- La riga di indice `candidato_pozzo` deve essere interamente composta da 0, in quanto dal pozzo non devono partire archi.
- La colonna di indice `candidato_pozzo` deve essere interamente composta da 1 (fatta eccezione per l'elemento `G[candidato][candidato]`, in quanto elemento della diagonale e dato che non stiamo ammettendo archi riflessivi).

```
1   def trovaPozzo(G):
2       i, j = 1, 2
3       candidato = 1
4       n = len(G)
5
6       while i <= n and j <= n:
7           if G[i][j] == 0:
8               j += 1
9               candidato = i
10          else:
```

```
11            i += 1
12            candidato = j
13
14        for k in range(1, n + 1):
15          if candidato == k:
16            continue
17          if G[k][candidato] != 1:
18            return null
19          if G[candidato][k] != 0:
20            return null
21
22        return candidato
```

Spiegazione algoritmo 1.7.5: Ricerca di un pozzo in tempo $O(n)$

1. **Fase di eliminazione (righe 2-13)**:
   - Vengono inizializzati due indici $i = 1$ e $j = 2$ per scorrere la matrice di adiacenza, e una variabile `candidato = 1` per memorizzare il nodo che attualmente potrebbe essere il pozzo.
   - Nel ciclo `while`, finché entrambi gli indici sono entro i limiti della matrice ($i \leq n$ e $j \leq n$), si effettua il controllo sulla cella $G[i][j]$.
   - Se $G[i][j]$ == 0, escludiamo j incrementando il suo indice (j++) e impostiamo il nuovo `candidato` a i.
   - Se $G[i][j]$ == 1, escludiamo i incrementando il suo indice (i++) e impostiamo il nuovo `candidato` a j.
   - Al termine del ciclo, la variabile `candidato` conterrà l'unico nodo superstite che può effettivamente essere un pozzo universale.
2. **Fase di verifica (righe 15-21)**:
   - Poiché la prima fase garantisce solo che `candidato` sia l'unico possibile pozzo (ma non che lo sia effettivamente), è necessario effettuare una verifica finale in tempo $O(n)$.
   - Si scorre ogni nodo k del grafo (escludendo il candidato stesso):
     - Si controlla che esista l'arco da k al candidato ($G[k]$[`candidato`] == 1). Se manca anche solo un arco entrante, l'algoritmo fallisce ritornando `null`.

  - Si controlla che non esista alcun arco dal candidato a k ($G[\texttt{candidato}][k] == 0$). Se viene trovato un arco uscente, l'algoritmo fallisce ritornando `null`.
- Se tutti i controlli hanno esito positivo, il `candidato` è confermato come pozzo universale e viene restituito (riga 23).

1.8 Ponti in un Grafo

Definizione 1.8.1: Ponte

Sia G un grafo connesso non diretto. Un arco $e \in E(G)$ è detto **ponte** se **non esiste un ciclo C che contiene** e.

A titolo esemplificativo, consideriamo la seguente rappresentazione grafica:

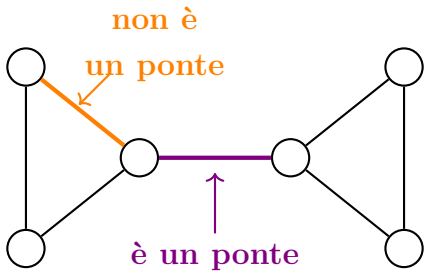


Figura 1.6: Esempio di archi ponte e non.

Osservazione 1.8.1

Se G è un albero, tutti gli archi sono ponti (perché un albero è un grafo connesso aciclico).

Esercizio 1.8.1: Stabilire se l'arco xy è un ponte

Consideriamo il seguente problema: *dato un grafo G e un arco xy, stabilire se xy è un ponte.*
Per risolverlo, possiamo procedere **rimuovendo l'arco** xy dal grafo: **se esiste comunque un cammino** da x a y, **significa che originariamente vi era un ciclo che comprendeva quell'arco** e dunque xy **non** può essere un ponte.

Il costo computazionale di questo algoritmo è legato alla ricerca di un cammino, e corrisponde pertanto a quello di una visita DFS, ovvero $\mathcal{O}(n+m)$.

Se volessimo estendere la procedura per trovare *tutti* i ponti di un grafo, l'idea di base si sviluppa nei seguenti passi:

1. Eseguiamo una visita DFS per ottenere l'albero delle visite T.
2. Per ogni arco $xy \in E(T)$, applichiamo la soluzione precedente: rimuoviamo l'arco e verifichiamo, tramite una DFS a partire da x, se riusciamo a raggiungere y.

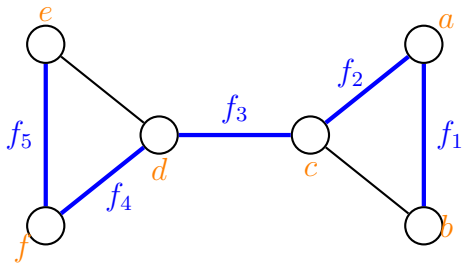
Il costo computazionale di questo approccio diventa $\mathcal{O}(m \cdot (n+m))$, dal momento che dobbiamo eseguire una DFS distinta per ogni arco dell'albero delle visite.

Osservazione 1.8.2

Sia G un grafo connesso e T un albero della DFS. Sia xy un arco di T, con x discendente di y. Sia T_y il sottoalbero dei discendenti di y. Se esiste un arco $uv \in E(G)$, $uv \neq xy$ con $u \in T_y$ e v un antenato di $x \implies xy$ non è un ponte.

L'osservazione risulta particolarmente intuitiva in quanto se esiste un arco NEL GRAFO (attenzione: l'albero T è un sottoinsieme del grafo, e poi stiamo cercando i ponti del grafo G, non di T) che connette un nodo sotto ad y con uno sopra al suo antenato x, quello sarà un arco all'indietro. Esistendo tale arco all'indietro vuol dire che esiste un ciclo e dunque xy non può essere un ponte.

Prendendo in esempio la seguente figura, l'arco (d, f) non può essere un ponte in quanto il nodo e risulta essere antenato di d, che però è connesso ad f.



Proposizione 1.8.1: Caratterizzazione dei ponti in un albero DFS

Sia G un grafo connesso e T un albero di ricerca DFS. Siano $xy \in E(T)$ con y discendente di x.

Allora esiste un arco $(u, v) \in E(G)$ con $u \in T_y$ e v antenato di $x \iff (x, y)$ **non è un ponte**.

Questa proposizione sostanzialmente completa la precedente, collegando i due concetti di arco all'indietro e ponte mediante una doppia implicazione.

Fatte le dovute osservazioni teoriche, possiamo procedere con la stesura dell'algoritmo.

```
def trovaPonti(G):
  ponti = []
  t = [0] * n
  c = 0
  dfs_ric(G, x, x, t, ponti)
  return ponti

def dfs_ponti(G, u: nodo, z: nodo, t, c, ponti):
  c += 1
  t[u] = c
  back = t[u]

  for v in u.neighbr:
    if t[v] == 0:
      b = dfs_ric(G, v, u, t, c, ponti)
      if b >= t[v]:
        ponti.add((u, v))
      if b < back:
        back = b
    else:
      if t[v] < back AND t[v] != t[z]:
        back = t[v]

  return back
```

Listing 1.9: Algoritmo per la ricerca dei ponti in un grafo.

Spiegazione algoritmo 1.8.1: Ricerca di tutti i ponti in un grafo

- **Inizializzazione (`trovaPonti`):**

1. `ponti = []`: Inizializziamo una lista vuota per raccogliere tutti gli archi che risulteranno essere ponti.
2. `t = [0] * n`: Array che tiene traccia di tutte le prime volte in cui un nodo è stato visitato (tempi di scoperta).
3. `c = 0`: Contatore globale per tenere traccia di ogni nodo visitato la prima volta.

- **Procedura ricorsiva (`dfs_ponti`):** La funzione riceve in input il grafo G, il nuovo vertice corrente u, il padre z da cui siamo arrivati, l'array dei tempi t, il contatore c e la lista dei `ponti`.

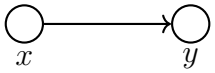
  1. `c += 1` e `t[u] = c`: Incrementiamo il contatore e registriamo il tempo di scoperta del nodo corrente u.
  2. `back = t[u]`: Teniamo traccia del nodo più indietro a cui possiamo arrivare tramite archi all'indietro.
  3. **Esplorazione dei vicini (`for v in u.neighbr`):**
     - **Se il nodo v non è visitato (`t[v] == 0`):**
       * `b = dfs_ric(G, v, u, t, c, ponti)`: Ci prendiamo il valore `back` ottenuto dalla DFS a partire dal nodo v.
       * **Condizione di ponte (`b >= t[v]`):** Se questo è vero, vuol dire che partendo da v siamo arrivati al massimo a v stesso o dopo, perché b indica al massimo quanto indietro possiamo andare partendo da v. Non esistono dunque vie alternative per risalire a un antenato di u, il che rende l'arco (u, v) un ponte. Di conseguenza aggiungiamo l'arco alla lista: `ponti.add((u, v))`.
       * **Aggiornamento del back (`b < back`):** Se risultiamo avere $b < back$ vuol dire che siamo andati ancora più indietro rispetto a dove siamo partiti, dunque l'arco corrente **non** può essere un ponte. A questo punto però sappiamo che il massimo indietro a cui possiamo andare a partire dal nodo corrente corrisponde a b e dunque aggiorniamo il valore di `back`.
     - **Se il nodo v è già stato visitato (`else`):**
       * **Controllo dell'arco all'indietro (`t[v] < back AND t[v] != t[z]`):** Se invece lo avessimo già visitato, se il nodo figlio v è stato scoperto di fatto prima del nodo corrente u (e non è il padre z da cui siamo arrivati), aggiorniamo il `back` con il tempo di scoperta di v (`back = t[v]`).

4. **Restituzione:** Alla fine dell'esplorazione, la funzione restituisce il valore di `back`.

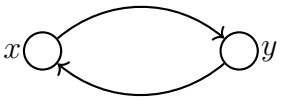
1.9 Grafi e componenti fortemente connesse

Definizione 1.9.1: Grafo fortemente connesso

Un grafo diretto si dice **fortemente connesso** se $\forall x, y \in V(G)$ esiste un cammino diretto $x \to y$ e un cammino da $y \to x$.



NON FORT. CONNESSO



FORTEMENTE CONNESSO

Osservazione 1.9.1

Per qualsiasi grafo diretto e $x \in V(G)$, esiste un sottografo fortemente connesso che contiene x.

Dimostrazione 1.9.1

Il sottografo consistente del solo nodo $\{x\}$ da solo è fortemente connesso, dunque sicuramente esiste.

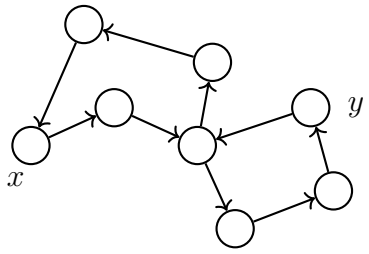
Definizione 1.9.2: Componente fortemente connessa

Una **componente fortemente connessa** è un **sottografo H fortemente connesso massimale (il più grande possibile con quei nodi)**. Formalmente, se $\exists H \subseteq H' \subseteq G$, allora H' non è più fortemente connesso.

Per questo motivo, due componenti non possono avere un vertice comune, altrimenti non sarebbe la massimale.

Ci si potrebbe chiedere: se x, y sono vertici della stessa componente fortemente connessa, esiste allora un ciclo che contiene x e y?

La risposta è **no**. Vediamo un controesempio:



Nota: quando ci viene chiesto se "allora esiste", si intende "allora **esiste sempre**".

Al contrario, vale la seguente proprietà: se esiste un ciclo C diretto che contiene x e y, ciò implica che x e y fanno parte della stessa componente fortemente connessa?

La risposta è, in quest'altro caso, **affermativa**.

Osservazione 1.9.2

Ogni vertice è singolarmente una componente fortemente connessa (ricordiamo $\{x\}$ essere una componente valida). Due componenti fortemente connesse sono sicuramente disgiunte (per massimalità) e dunque **le componenti costituiscono una partizione dei vertici del grafo**.

A titolo esemplificativo, consideriamo la seguente scomposizione in componenti fortemente connesse:

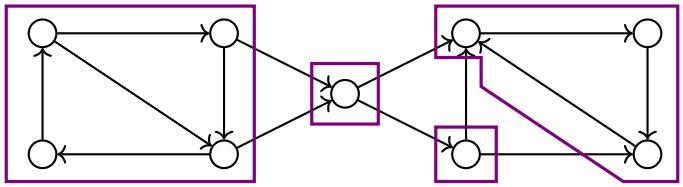


Figura 1.7: Esempio grafico di come le componenti di un grafo diretto ne costituiscano una partizione.

Ora abbiamo qualcosa con cui giocare: **trovare tutte le componenti fortemente connesse di un grafo**.

Proposizione 1.9.1: Caratterizzazione di grafo fort. conn.

Sia G un grafo diretto e $u \in V(G)$.

$$G \text{ è fort. conn.} \iff \forall x \in V(G) \,\exists \text{ un cammino } x \to u \text{ e un cammino } u \to x$$

1. Se il grafo non ha cicli, ogni singolo nodo è una componente fortemente connessa.

2. Dobbiamo poi applicare una tecnica chiamata **contrazione**.

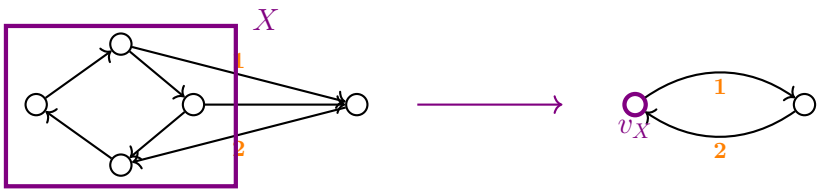
   Prendiamo $X \subset V(G)$ e sostituiamo quell'insieme di vertici con un unico nuovo vertice v_x. Avendo rimosso tutti i $v \in X$, dobbiamo anche rimuovere tutti gli archi del tipo xy con $x, y \in X$ (gli archi interni a quella porzione di grafo), ma **mantenere tutti gli archi che arrivano a v_x dall'esterno** ((y, x) t.c. $y \notin X, x \in X$) **o che vanno verso l'esterno** ((x, y) t.c. $x \in X, y \notin X$).

Proposizione 1.9.2: Contrazione di un sottografo fortemente connesso

Se G è un grafo fortemente connesso e $H \subseteq G$ è un sottografo fortemente connesso, allora il grafo quoziente $G/V(H)$ ottenuto contraendo H in un unico vertice è anch'esso fortemente connesso.

La notazione $G/V(H)$ indica il nuovo grafo ottenuto dopo la contrazione. Se volessimo definirlo formalmente:

$$G/V(H) = (G - H) \cup v_H$$



Presentiamo dunque ora un primo algoritmo (inefficiente) per l'estrazione di tutte le componenti fortemente connesse di un grafo.

```
def Fort(G):
  C = trova_ciclo(G)

  if not C:
    return {v_1, v_2, ..., v_n}

  G = G/C

  {u_1, ..., u_k} = Fort(G)

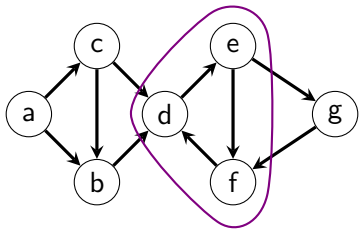
  for i in range(1, k):
    if v_c in u_i:
      u_i = (u_i - {v_c}) U V(C) #1
```

Spiegazione algoritmo 1.9.1: Estrazione componenti fort. connesse

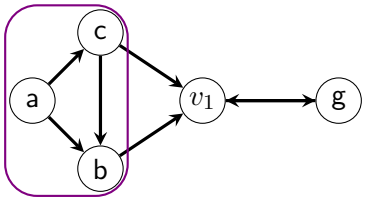
Analizziamo passo dopo passo il funzionamento dell'algoritmo:

1. **Ricerca del ciclo**: L'algoritmo inizia cercando un ciclo C all'interno del grafo diretto G.
2. **Caso base**: Se il grafo non contiene alcun ciclo, significa che ogni vertice costituisce una componente fortemente connessa a sé stante, e l'algoritmo restituisce l'insieme di tutti i vertici $\{v_1, v_2, \ldots, v_n\}$.
3. **Contrazione**: Se viene trovato un ciclo C, il grafo viene contratto mediante l'operazione $G = G/C$, collassando i vertici del ciclo in un unico super-vertice v_c.
4. **Chiamata ricorsiva**: Si invoca nuovamente la funzione ricorsiva sul grafo contratto per ottenere le componenti fortemente connesse $\{u_1, \ldots, u_k\}$.
5. **Fase di unione (unpacking)**: Infine, si esamina ciascuna componente restituita: se il vertice di contrazione v_c appartiene alla componente u_i, esso viene sostituito dall'insieme di tutti i vertici originali che componevano il ciclo C da cui è stato ottenuto il vertice v_c, ripristinando la struttura iniziale.

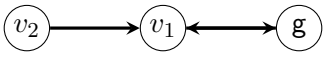
A titolo esemplificativo, consideriamo il seguente grafo orientato e seguiamo l'evoluzione dell'algoritmo:



Passo 1: Troviamo il ciclo $C_1 = \{d, e, f\}$ e lo sostituiamo con il super-vertice v_1. Il grafo contratto appare come segue:



Passo 2: Troviamo il secondo ciclo $C_2 = \{a, b, c\}$ e lo sostituiamo con il super-vertice v_2.



Passo 3: Siccome non abbiamo più cicli, iniziamo la fase di *unpacking* (quella marcata con #1 nel codice), espandendo i super-vertici precedentemente contratti per ritornare alla rappresentazione originale delle componenti fortemente connesse.

Proposizione 1.9.3: Proprietà degli archi di attraversamento

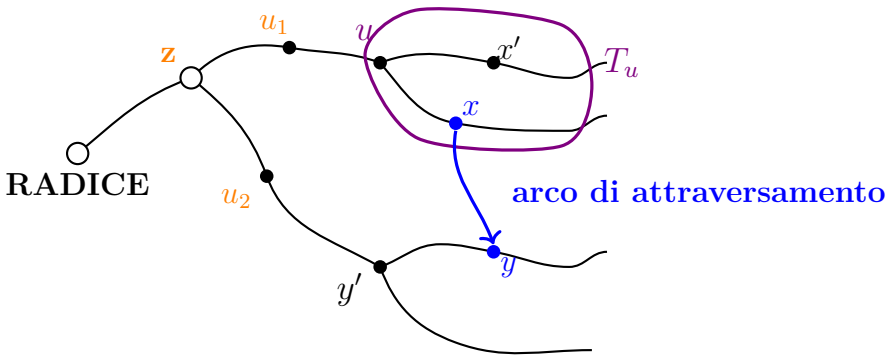
Sia T l'albero di ricerca DFS su un grafo diretto, sia $u \in V(G)$ e sia (x, y) un arco di attraversamento tale che $x \in T_u$ e $y \notin T_u$.
Allora esiste un vertice $z \in V(G)$, antenato comune più vicino (LCA) di x e y, tale che:

- z è un antenato di u;
- esistono due figli distinti u_1, u_2 di z tali che:

$$x \in T_{u_1} \qquad \text{e} \qquad y \in T_{u_2}$$

- per ogni $x' \in T_{u_1}$ e per ogni $y' \in T_{u_2}$, vale la relazione sui tempi di scoperta:

$$t(x') > t(y')$$



Intuitivamente, dall'analisi della struttura ad albero, è immediato osservare che tutti i nodi appartenenti a T_{u_2} vengono visitati **prima** di quelli appartenenti a T_{u_1}, dato che l'arco di attraversamento è orientato da $x \in T_{u_1}$ verso $y \in T_{u_2}$.

Definizione 1.9.3: C-radice

Sia C una componente fortemente connessa di un grafo diretto G e sia T un'arborescenza prodotta da una visita DFS su G.
Si definisce **C-radice** il primo nodo appartenente a C che viene visitato dalla DFS.

Proposizione 1.9.4: Proprietà delle C-radici

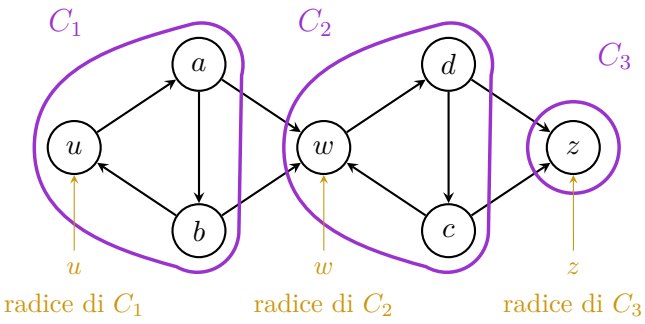
Sia u una C-radice all'interno di un'arborescenza DFS T. Allora valgono le seguenti proprietà:

1. $C(u) \subseteq T_u$, dove la notazione $C(u)$ indica l'intera componente fortemente connessa che contiene il vertice u.

2. Se $u_1, u_2, \ldots, u_k$ sono tutte le C-radici contenute nel sottoalbero T_u, allora l'insieme dei vertici di T_u è dato dall'unione disgiunta delle relative componenti fortemente connesse:

$$V(T_u) = C(u_1) \cup C(u_2) \cup \cdots \cup C(u_k)$$

Prendendo come esempio il grafo di seguito riportato, verifichiamo intuitivamente la validità delle proprietà enunciate.



- **Verifica del Punto 1:**

  Prendiamo u come radice dell'arborescenza T_u e supponiamo che la visita DFS esplori i vertici nel seguente ordine:

  $$T_u = \{u, a, w, d, z, c, b\}$$

  Si verifica immediatamente la validità della proprietà, in quanto:

  $$C_1 \subseteq T_u$$

  Considerando invece il vertice w, l'arborescenza generata dalla DFS a partire da w risulta:

  $$T_w = \{w, d, z, c\}$$

  Anche in questo caso vale banalmente:

  $$C(w) = C_2 \subseteq T_w$$

  La proprietà risulta verificata in modo immediato anche per il terzo caso ($T_z = \{z\} = C_3$).

- **Verifica del Punto 2:**

  Consideriamo ad esempio l'arborescenza $T_w = \{w, d, z, c\}$. La proprietà afferma che l'arborescenza generata dalla DFS a partire da un nodo può essere ottenuta come l'unione delle componenti fortemente connesse a cui appartiene ciascun nodo raggiunto dalla visita.

  Determiniamo la componente fortemente connessa di appartenenza per ogni vertice in T_w:

  $$C(w) = \{w, d, c\} \qquad C(c) = \{w, d, c\}$$
  $$C(d) = \{w, d, c\} \qquad C(z) = \{z\}$$

  Calcolando l'unione di tali componenti:

  $$T_w = \underbrace{C(w) \cup C(d) \cup C(c)}_{*} \cup C(z) = \{w, d, c\} \cup \{z\} = \{w, d, z, c\}$$

  (*) Essendo $C(w) = C(d) = C(c)$, i duplicati non alterano il risultato dell'unione insiemistica.

  L'uguaglianza risulta pertanto pienamente confermata.

Supponendo di disporre di un metodo per riconoscere le C**-radici** (ovvero le radici delle componenti fortemente connesse), è possibile utilizzare una visita in profondità (DFS) modificata per individuare ed estrarre le componenti fortemente connesse (SCC) di un grafo orientato.

```
    def DFS_SCC(G, u, open, output, vis):
      vis[u] = 1
      open.push(u)

      for v in u.neighbor:
        if vis[v] == 0:
          DFS_SCC(G, v, open, output, vis)

      if u è una C-radice(u):
        C = []
        do:
          w = open.pop()
          C.append(w)
        while w != u

      output.add(C)

    def SCC(G):
      open = Stack()
      output = set()
      vis = [0] * n

      for v in G:
        if vis[v] == 0:
          DFS_SCC(G, v, open, output, vis)

      return output
```

Listing 1.10: Propotipo di algoritmi efficiente per la ricerca delle componenti fortemente connesse

Spiegazione algoritmo 1.9.2: Ricerca componenti fortemente connesse

L'algoritmo si struttura in una funzione principale `SCC(G)` e in una procedura ausiliaria ricorsiva `DFS_SCC`.

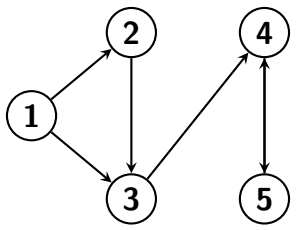
Ruolo dei parametri e delle variabili:

- `u`: rappresenta il **nodo corrente** in fase di esame.
- `open`: è una pila (*stack*) utilizzata per tenere traccia dei nodi visitati ma i cui sottoalberi non sono stati ancora "chiusi", ovvero le cui componenti non sono ancora state completate.
- `output`: la struttura dati contenente l'insieme di tutte le componenti fortemente connesse identificate.
- `vis`: il vettore booleano di marcatura per i nodi visitati.

Spiegazione dettagliata del funzionamento:

1. **Inizializzazione (`SCC`)**: Viene creato uno stack `open` vuoto, un insieme `output` per contenere le componenti individuate e il vettore `vis` di dimensione n azzerato. Per ogni vertice non ancora visitato, si avvia la procedura ricorsiva `DFS_SCC`.
2. **Visita e tracciamento (`DFS_SCC`)**: All'ingresso nella procedura, il nodo u viene marcato come visitato (`vis[u] = 1`) e inserito nello stack `open`. Successivamente, si esplorano ricorsivamente tutti i vicini v non visitati.
3. **Identificazione e chiusura della componente**: Una volta terminata l'esplorazione dei vicini di u, si verifica se u è una C-radice (supponendo di conoscere un modo efficiente per determinare questa condizione).
   - Se u è una C-radice, si definisce una nuova lista C, la quale rappresenterà esattamente la componente fortemente connessa di cui u è la radice.
   - Si esegue un ciclo `do-while` che rimuove mediante `pop()` tutti i nodi dallo stack `open` fino a incontrare il nodo u incluso. Questo ciclo si occupa sostanzialmente di estrarre tutti i nodi che nello stack si trovano al di sopra della C-radice u, poiché essi vi sono stati inseriti successivamente e appartengono necessariamente alla medesima componente.
   - Infine, la componente C così costruita viene aggiunta alla collezione globale `output`.

Vediamo un esempio con il seguente grafo:



1. **Passo 1-5:** Visitiamo con DFS il grafo, ottenendo ciò:

   $$\text{open} = [1, 2, 3, 4, 5], \qquad \text{output} = []$$

   A questo punto da 5 si dovrebbe andare ad un nodo non visitato. Siccome non ne abbiamo altri, durante la visita di 5 si esce dal ciclo `for` e si passa al controllo. 5 non è una C-radice e dunque si esce senza restituire nulla.

2. **Passo 6:** Risaliamo nella ricorsione tornando a 4. Anche qui non possiamo visitare nuovi nodi dunque controllo. In questo caso 4 è una C-radice, dunque eseguiamo il `while`.

   In questo caso rimuoviamo solo 5 che è l'unico nodo sopra a 4 (oltre a 4 stesso) e li mettiamo nella stessa componente.

   Nota: Il `do-while` si ferma appena rimuoviamo la C-radice.

   $$\text{open} = [1, 2, 3], \qquad \text{output} = [[4, 5]]$$

3. **Passo 7-8:** Si sale nella ricorsione, prima a 3 e poi a 2. Siccome nessuna delle due è una C-radice si ritorna [].

4. **Passo 9:** Arrivati ad 1 e avendo visitato tutti i nodi, finiamo nel controllo. In questo caso siamo su una C-radice, dunque procediamo col rimuovere tutti i nodi sopra di essa, fermandoci quando avremo rimosso anche lei.

   Siccome 1 era l'ultimo nodo, l'algoritmo termina col seguente output:

   $$\text{output} = [[5, 4], [3, 2, 1]]$$

Abbiamo dato per assunto di saper riconoscere una C-rad in modo abbastanza semplice. Formalizziamo.

Proposizione 1.9.5: Caratterizzazione di C-radice

Un nodo u non è una C-radice $\iff$ dalle chiamate ricorsive `DFS_SCC` con radice u, viene attraversato un arco (v, w) t.c. w è stato già visitato ($t(w) < t(u)$) ma la componente di w non è stato stabilito ($w \in$ open, è ancora nello stack).

In parole molto povere: affinché un nodo sia C-radice deve necessariamente essere il più vecchio (quello scoperto prima di tutto il suo gruppo).

Inseriamo nel codice una variabile `back` che è il più all'indietro a cui possiamo arrivare nelle chiamate a partire da u.

Definiamo anche un nuovo array CC[] dove CC[v] vale:

- 0: se v non è stato visitato
- $-t[v]$: se è stato visitato e sta nello stack
- C: valore positivo che indica il numero della componente a cui appartiene e dunque non sta nello stack.

Stiamo adottando una logica analoga a quella impiegata per l'individuazione dei ponti all'interno di un grafo.

```
    def DFS_SCC(G, u, CC, open, t, comp):
      t += 1
      CC[u] = -t
      open.push(u)
      back = t

      for v in u.neighbr:
        if CC[v] == 0:
          b = DFS_SCC(G, v, CC, open, t, comp)
          back = min(b, back)
        if CC[v] < 0:
          back = min(back, -CC[v])

      if back == -CC[u]:
        comp += 1
        while True:
          w = open.pop()
```

```
            CC[w] = comp
            if w == u:
              break

        return back

    def SCC(G):
        CC = [0] * n
        open = Stack()
        t = 0
        comp = 0

        for u in G:
          if CC[u] == 0:
            DFS_SCC(G, u, CC, open, t, comp)

    return CC
```

Listing 1.11: Algoritmo di Tarjan per la ricerca delle componenti fortemente connesse

Spiegazione algoritmo 1.9.3: Algoritmo di Tarjan

- **Inizializzazione del nodo (`CC[u] = -t`)**: Il contatore globale del tempo t viene incrementato e assegnato a u. Memorizziamo temporaneamente $-t$ nell'array `CC` per indicare sia che il vertice è attualmente presente nello stack (`open`), sia l'istante esatto della sua scoperta.
- **Inizializzazione di `back`**: Inizialmente, il nodo raggiungibile più indietro nel percorso coincide con il nodo stesso, per cui poniamo `back` $= t$.
- **Esplorazione dei vicini**: Per ciascun vertice adiacente $v \in u$.neighbr:
  - **Nodo non visitato (`CC[v] == 0`)**: Si visita ricorsivamente v. La chiamata restituisce il valore minimo di scoperta raggiungibile dal sottoalbero di v, permettendo di aggiornare `back` $= \min(b, \texttt{back})$.
  - **Arco all'indietro o trasversale verso nodo attivo (`CC[v] < 0`)**: Se v è già stato visitato ma appartiene ancora allo stack `open` (segnalato dal valore negativo di `CC[v]`), significa che v appartiene alla componente corrente. Aggiorniamo quindi il valore tramite `back` $= \min(\texttt{back}, -\texttt{CC}[v])$.

- **Identificazione della C-radice ed estrazione**: Se `back` == $-$`CC`$[u]$, significa che dal sottoalbero radicato in u non è possibile risalire ad alcun antenato scoperto prima di u. Di conseguenza, u è la **C-radice** della componente. Incrementiamo il contatore `comp` ed estraiamo gli elementi dalla pila `open` fino a rimuovere u stesso, assegnando a ciascuno di essi l'identificativo definitivo della componente (`CC[w] = comp`).

Osservazione 1.9.3: Funzionamento dell'algoritmo

In base alla proprietà precedentemente enunciata, un nodo è una C-radice se e solo se è il primo vertice della componente connessa ad essere scoperto durante la visita DFS. Da tale vertice in avanti esisteranno esclusivamente archi discendenti nell'arborescenza di visita. Per tale motivo, la soluzione si basa sulla medesima strategia di tracciamento del valore `back` impiegata per la ricerca dei ponti.

1.10 BFS - Breadth-First-Search

Consideriamo il seguente problema: dato un grafo non orientato $G = (V, E)$, si vuole trovare la **distanza** da un vertice sorgente x a un vertice y.

Definizione 1.10.1: Distanza tra due vertici

La **distanza** $\text{dist}(x, y)$ tra due vertici x e y è definita come la **lunghezza del minimo cammino** da x a y.

Osservazione 1.10.1: Lunghezza di un cammino

La **lunghezza** è il numero di archi presenti nel cammino da x a y.

Risolvere questo problema per una singola coppia equivale a risolvere un problema più generale: trovare $\text{dist}(x, z)$ per ogni $z \in V(G)$.

Possiamo subito individuare due casi base banali:

- $\text{dist}(x, x) = 0$
- $\text{dist}(x, w) = 1$, per ogni vertice w adiacente a x ($w \in x.\text{neighbr}$)

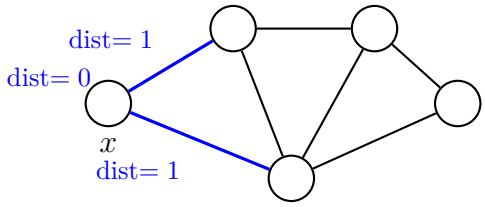


Figura 1.8: Esempio di distanze appuntate per i nodi rispettanti i casi base.

Avendo individuato questi casi base, possiamo scalare il problema progressivamente a più nodi, ottenendo così l'algoritmo di visita in ampiezza **Bredth-First-Search (BFS)**.

```
def BFS(G, x):
  dist = [-1] * n
  dist[x] = 0

  Q = Queue()
  Q.push(x)

  while Q.size() != 0:
    y = Q.pop()
    for z in y.neighbr:
      if dist[z] == -1:
        dist[z] = dist[y] + 1
        Q.push(z)

  return dist
```

Listing 1.12: Algoritmo BFS per il calcolo delle distanze da un nodo sorgente

Spiegazione algoritmo 1.10.1: BFS

Analizziamo nel dettaglio le istruzioni dell'algoritmo:

- `dist = [-1] * n`: Array che memorizza tutte le distanze dei vertici del grafo rispetto alla sorgente x. Dato che il valore 0 è riservato alla distanza dal vertice stesso, inizializziamo tutti i valori a -1 per indicare che i nodi non sono stati ancora visitati.
- `dist[x] = 0`: Impostiamo la distanza del nodo sorgente x a 0, in virtù del caso base analizzato in precedenza.

- `Q = Queue()` e `Q.push(x)`: Creando una coda FIFO (*First In, First Out*), abbiamo modo di tenere traccia mano a mano di tutti gli adiacenti e solo successivamente passare agli adiacenti degli adiacenti (garantendo un'esplorazione a livelli, e non andando in profondità come in un DFS classico).
- `y = Q.pop()`: Rimuoviamo l'elemento in testa alla coda per esaminarne i vicini adiacenti.
- `if dist[z] == -1`: Verifichiamo se il vertice z è già stato visitato. Se fosse già stata segnata una distanza per quel vertice, rischieremmo di sovrascrivere la distanza minima di un nodo già visitato in precedenza.
- `dist[z] = dist[y] + 1`: Essendo z adiacente a y ($z \sim y$), la sua distanza dalla sorgente sarà pari alla distanza di y incrementata di 1.
- `Q.push(z)`: Infine, aggiungiamo z alla coda per poter analizzare i suoi vicini nelle iterazioni successive.

Analizzando la complessità, otteniamo che:

- Il ciclo `while` viene eseguito $\forall y \in V(G)$.
- Dentro al ciclo eseguiamo un lavoro pari al grado del vertice.
- Il tutto è descritto dall'equazione:

$$\sum_{y \in V(G)} (1 + \deg(y)) = \mathcal{O}(\mathbf{n} + \mathbf{m})$$

Proposizione 1.10.1: Esistenza di un adiacente a distanza inferiore

Sia G un grafo e siano x, y due suoi vertici. Se $\text{dist}(x, y) = k$, allora $\exists$ un vicino z di y con $\text{dist}(x, z) = k - 1$.

Non verrà fornita una dimostrazione formale per la proposizione, ma intuitivamente, se è definita la distanza da $x \to y$, significa che esiste un cammino $x \to y$. Dunque ci sarà un nodo appartenente a tale cammino che viene subito prima di y, e tale nodo è proprio z.

Usiamo questa proposizione per dimostrare la correttezza dell'algoritmo BFS, cioè $\text{dist}[y] = \text{dist}(x, y)$, $\forall y$:

- Per x abbiamo $\text{dist}[x] = 0 = \text{dist}(x, x)$.

- Per i nodi adiacenti ad x vediamo che la distanza viene correttamente impostata a $\text{dist}(x, x) + 1 = 1$.
- Usiamo la proposizione, dunque sappiamo che $\exists y$ t.c. $\text{dist}(x, y) = k$.

  Esiste dunque $z \sim y$ con $\text{dist}(x, z) = k - 1$:

  $\implies$ $\text{dist}[z] = k - 1$

  $\implies$ Essendo z visitato prima di y ed essendo $z \sim y$, y verrà visitato all'iterazione dopo.

  Dunque:

$$\text{dist}(x, y) = \text{dist}(x, z) + \text{dist}(z, y) = k - 1 + 1 = k$$

1.10.1 Estrazione del cammino minimo

Abbiamo dimostrato come l'algoritmo BFS ci permetta di trovare correttamente la distanza di un nodo z da un nodo radice x. E ricordiamo la distanza essere la lunghezza del cammino minimo. Ma come possiamo trovare questo cammino minimo, ossia l'insieme dei nodi (e archi) che ci permettono di arrivare da $x \to z$?

Osservazione 1.10.2: Numero di cammini minimi in un grafo

In un grafo con n vertici possono esistere fino a circa $2^{n/2}$ cammini minimi distinti. Di conseguenza, non è possibile elencare individualmente tutti i cammini in tempo polinomiale; tuttavia, è possibile determinarne efficientemente il numero totale.

Di seguito viene presentata una versione modificata della BFS che calcola la distanza minima, ricostruisce l'albero di visita tramite il vettore dei padri e conta esattamente quanti cammini minimi esistono da x verso qualsiasi altro vertice del grafo.

```
1     def BFS_comp(G, x):
2       dist = [-1] * n
3       count = [0] * n
4       P = [None] * n
5
6       Q = Queue()
7       Q.push(x)
8
9       dist[x] = 0
10      count[x] = 1
```

```
11        P[x] = x
12
13        while Q.size() != 0:
14          y = Q.pop()
15          for z in y.neighbr:
16            if dist[z] == -1:
17              dist[z] = dist[y] + 1
18              Q.push(z)
19              count[z] = count[y]
20              P[z] = y
21            else:
22              if dist[z] == dist[y] + 1:
23                count[z] += count[y]
24
25        return count, dist, P
```

Listing 1.13: Algoritmo BFS per il conteggio dei cammini minimi

Spiegazione algoritmo 1.10.2: Conteggio dei cammini minimi con BFS

Analizziamo nel dettaglio le strutture dati utilizzate e il funzionamento dell'algoritmo:

- **Inizializzazione delle strutture dati**:
  - `dist`: vettore di dimensione n inizializzato a -1 per tenere traccia delle distanze minime dalla radice x. Il valore -1 indica che il nodo non è ancora stato visitato.
  - `count`: array che tiene traccia, per ogni nodo del grafo, del numero di cammini minimi per arrivarci a partire da x. Inizialmente tutti i valori sono impostati a 0, in quanto nessun cammino è ancora stato trovato.
  - `P`: vettore dei padri impiegato per memorizzare la struttura dell'albero di visita.
- **Caso base (inizializzazione della radice)**: Per il nodo sorgente x, si imposta `dist[x]` $= 0$ e `P[x]` $= x$ (inizializzazione della radice nel vettore dei padri). Inoltre, si imposta `count[x]` $= 1$, poiché esiste un unico cammino (di lunghezza 0) che collega x a se stesso.

- **Esplorazione dei vicini**: Durante l'estrazione di un nodo y dalla coda Q, si esamina ogni suo vicino $z \in$ `y.neighbr`:

  - **Caso 1: Nodo z non ancora visitato (`dist[z] == -1`)**:
    Se non abbiamo ancora visitato z, lo impostiamo come visitato assegnandogli la distanza `dist[z]` $=$ `dist[y]` $+ 1$, lo inseriamo nella coda di esplorazione Q e aggiorniamo il suo padre `P[z]` $= y$.
    Fondamentale: siccome z non era ancora visitato e lo stiamo scoprendo a partire da y, significa che z appartiene allo stesso cammino minimo che arriva a y. Di conseguenza, il conteggio dei cammini minimi per raggiungere z eredita esattamente il valore del conteggio del nodo y:

    $$\texttt{count[z]} = \texttt{count[y]}$$

  - **Caso 2: Nodo z già visitato (`else`)**:
    Se invece z è un nodo adiacente a y ma è già stato visitato in precedenza, procediamo in modo diverso. In particolare, verifichiamo se vale la condizione:

    $$\texttt{dist[z]} == \texttt{dist[y]} + 1$$

    Se tale uguaglianza è verificata, significa che z si trova a una distanza da x esattamente pari a `dist[y]` $+ 1$. Ciò implica che z può essere raggiunto con un cammino minimo *anche* passando attraverso il nodo y. Di conseguenza, z può essere raggiunto da tutti i cammini minimi giunti fino a y, e aggiorniamo il totale sommando il contributo di y:

    $$\texttt{count[z]} \mathrel{+}= \texttt{count[y]}$$

Definizione 1.10.2: Distanza tra due sottoinsiemi di vertici

Dati due sottoinsiemi di vertici $X, Y \subseteq V(G)$, si definisce la **distanza tra X e Y** come:

$$\text{dist}(X, Y) = \min\{\text{dist}(x, y) \mid x \in X,\ y \in Y\}$$

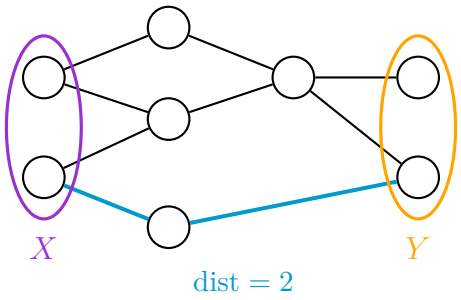


Figura 1.9: Cammino minimo tra i sottoinsiemi di nodi X e Y.

Vogliamo progettare un algoritmo efficiente per determinare la distanza minima che intercorre tra due sottoinsiemi di vertici X e Y all'interno di un grafo G.

Approccio Brute-Force. Una prima soluzione immediata consiste nell'eseguire una visita in ampiezza (BFS) a partire da ciascun elemento appartenente al sottoinsieme X verso tutti i vertici del sottoinsieme Y.

Questo metodo risulta molto semplice ed efficace, ma comporta un costo computazionale pari a:

$$\mathcal{O}(|X| \cdot (n + m)) = \mathcal{O}(n(n + m))$$

Soluzione Ottimizzata (BFS Multi-Sorgente). È possibile ottimizzare il procedimento facendo partire **"contemporaneamente"** la visita BFS da tutti i vertici $x \in X$.

Nel pratico questo risultato si ottiene inserendo inizialmente tutti i nodi di X all'interno della coda (`queue`) con distanza impostata a 0, per poi eseguire una normale esplorazione in ampiezza.

```
def BFS_insiemi(G, X, Y):
  dist = [-1] * n
  Q = queue()

  for x in X:
    dist[x] = 0
    Q.push(x)

  while Q.size() != 0:
    # Esecuzione della BFS classica
    current = Q.pop()
    for v in current.neighbor:
      if dist[v] == -1:
        dist[v] = dist[current] + 1
        Q.push(v)
```

Listing 1.14: Algoritmo BFS multi-sorgente per la distanza tra sottoinsiemi

Spiegazione algoritmo 1.10.3: Distanza tra sottoinsiemi

- **Inizializzazione delle strutture dati**: Viene creato un array delle distanze `dist` di dimensione n, inizializzato interamente al valore -1 per indicare che nessun nodo è stato ancora visitato. Contestualmente viene istanziata una coda vuota `Q`.
- **Caricamento delle sorgenti**: Si scorrono tutti i vertici appartenenti al sottoinsieme X. Per ciascun vertice $x \in X$, si imposta la sua distanza sorgente a 0 (`dist[x] = 0`) e lo si inserisce nella coda `Q`.
- **Propagazione a monte**: All'interno del ciclo `while Q.size() != 0`, si esegue l'estrazione e la visita standard della BFS. Poiché tutti i vertici dell'insieme X sono stati inseriti contemporaneamente all'inizio con distanza zero, la visita si espanderà come un unico "fronte d'onda" a livelli di distanza crescenti.
- **Correttezza ed efficienza**: Il primo vertice appartenente al sottoinsieme Y che viene estratto o scoperto durante l'esplorazione conterrà la distanza minima assoluta tra l'insieme X e l'insieme Y. La complessità temporale totale dell'algoritmo si riduce a quella di una singola visita in ampiezza, ovvero $\mathcal{O}(n + m)$.

Proposizione 1.10.2: Distanza tra vertici adiacenti in BFS

Sia `dist` l'array delle distanze a partire dal vertice x, generato da una visita in ampiezza (BFS). Allora per ogni coppia di vertici adiacenti y, z ($y \sim z$), si ha che:

$$|\operatorname{dist}(y) - \operatorname{dist}(z)| \leq 1$$

Anche per questa proposizione non verrà fornita una dimostrazione formale, tuttavia si invita il lettore a ragionare sull'intuitività di tale. Sappiamo che se $x \sim y$ allora `dist[y] = dist[x] + 1`, dunque vediamo già come queste due distanze varino di una sola unità. La ragione per cui si prende il modulo di tale differenza è per impedire casi in cui prendiamo come riferimento il nodo y, visitato dopo il nodo x, ma che non essendo in un grafo orientato risulta comunque adiacente e raggiungibile da x. A tal proposito, in modo che tale proposizione valga per ogni coppia di vertici, è corretto inserire l'operatore modulo. Inoltre, non si può mettere un $=$ stretto in quanto si considera anche il caso in cui $x = y$.

Proposizione 1.10.3: Uguaglianza tra alberi di visita BFS e DFS

Sia G un grafo non diretto e connesso, e sia $x \in V(G)$. Siano T_1 un albero BFS e T_2 un albero DFS del grafo G, entrambi aventi radice nel vertice x. Allora abbiamo che:

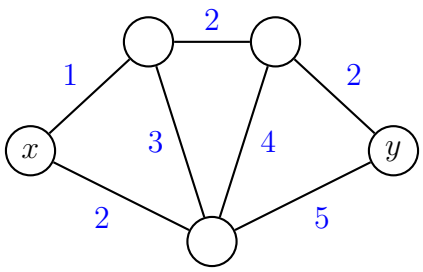
$$T_1 = T_2 \iff G \text{ è un albero}$$

1.11 Grafi pesati

Definizione 1.11.1: Grafo pesato

Un grafo si dice **pesato** se a ogni suo arco viene associato un **peso**, ovvero un numero reale positivo. Formalmente definiamo una funzione di peso:

$$w \colon E(G) \longrightarrow \mathbb{R}^+$$



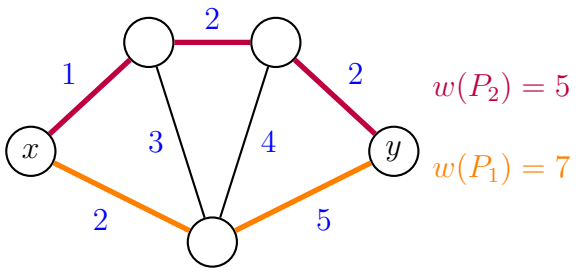
Definizione 1.11.2: Peso di un cammino e distanza pesata

Il **peso di un cammino** P è definito come la somma di tutti i pesi associati agli archi che lo compongono:

$$w(P) = \sum_{e \in P} w(e)$$

Analogamente a quanto fatto per i grafi non pesati, definiamo come **distanza pesata** $\text{dist}_w(x, y)$ tra due vertici x e y il peso del cammino di peso minimo che li collega:

$$\text{dist}_w(x, y) = \min_{\forall P \text{ tra } x \to y} w(P)$$



Osservazione 1.11.1: Proprietà della distanza pesata

La distanza pesata dist_w soddisfa le seguenti proprietà fondamentali:

- $\text{dist}_w(x, x) = 0$;
- $\text{dist}_w(x, y) \geq 0$, ammettendo l'uguaglianza se e solo se $y = x$, poiché tutti i pesi associati agli archi sono strettamente positivi per definizione;
- $\text{dist}_w(x, y) \leq \text{dist}_w(x, z) + \text{dist}_w(z, y)$.

Se il grafo è diretto, essenzialmente la definizione non cambia: $\text{dist}_w(x, y)$ rappresenta il peso minimo di un cammino **diretto** da x a y.

Osservazione 1.11.2: Confronto tra grafi orientati e non orientati

L'unica sostanziale differenza tra le due tipologie di grafo riguarda la simmetria della funzione distanza:

- **Nei grafi non diretti**: $\text{dist}_w(x, y) = \text{dist}_w(y, x)$ per ogni coppia di vertici.
- **Nei grafi diretti**: molto spesso si ha $\text{dist}_w(x, y) \neq \text{dist}_w(y, x)$.

1.11.1 Calcolo delle distanze pesate

Consideriamo ora il problema generale di determinare la distanza $\text{dist}_w(x, y)$ all'interno di un grafo non diretto. Se il peso di ogni arco è unitario, ovvero $w(e) = 1$ per ogni $e \in E(G)$, il problema si riduce semplicemente all'esecuzione di una visita in ampiezza (**BFS**).

Tuttavia, in presenza di pesi generici la situazione non è così semplice.
La distanza pesata da x a v non è più necessariamente pari al peso dell'arco diretto (x, v), poiché potrebbe esistere un cammino alternativo da x a v con un peso complessivo minore. Nel nostro esempio, il cammino più breve è quello che passa per il nodo u, fornendo un peso totale di $1 + 1 = 2$, che risulta inferiore a 5.

Tuttavia, possiamo stabilire un'importante certezza: partendo da x e considerando l'insieme dei suoi nodi adiacenti, se selezioniamo l'arco di **peso minimo** uscente da x (sia esso l'arco (x, u)), siamo matematicamente certi che la distanza pesata da x a u sarà esattamente pari al peso dell'arco stesso, ossia:

$$\text{dist}_w(x, u) = w((x, u))$$

Questo è garantito dal fatto che, essendo (x, u) l'arco uscente di peso minimo da x, non può esistere alcun altro cammino alternativo verso u con peso strettamente minore.

Però andando avanti non possiamo dire la stessa cosa: **non sempre scegliere l'arco di peso minimo è la scelta giusta, dato che potremmo avere un cammino a lungo andare molto pesante contro un arco diretto pesante ma non così tanto**.

Supponiamo di aver trovato un insieme Z di vertici per il quale conosciamo $\text{dist}_w(x, y)$ con $y \in Z$. Nel nostro esempio precedente, questo insieme sarà inizialmente composto solo da x e dal nodo con cui condivide l'arco di peso minore, che abbiamo chiamato u.

Proposizione 1.11.1: Estensione dell'insieme delle distanze minime

Se v è il vertice con arco (u', v) tale che $v \notin Z$, $u' \in Z$ e la quantità $w(u', v) + \text{dist}_w(x, u')$ è minima, allora:

$$\text{dist}_w(x, v) = \text{dist}_w(x, u') + w(u', v)$$

In parole molto povere, v è il nodo **fuori** da Z che ha un arco verso un nodo **dentro** Z e tale arco ci permette di minimizzare la distanza dalla sorgente x a tale nodo v.

In modo concettuale, proviamo a stendere un algoritmo:

1. Inizializziamo l'insieme Z contenente inizialmente solo la radice x.

2. Iteriamo lungo tutti gli adiacenti di x e prendiamo la loro distanza pesata da x.
3. Una volta scorsi tutti, prendiamo quello di peso minimo e lo aggiungiamo a Z.
4. Adesso Z è cresciuto, ma la procedura è la stessa: per ogni nodo $v \notin Z$, applichiamo il procedimento descritto sopra, che è valido in quanto ad ogni nodo di Z è associata una distanza calcolata rispetto al nodo sorgente x.

Tale algoritmo è il motore sotto il quale girano Google Maps e tutti i principali servizi di navigazione: è l'**algoritmo di Dijkstra**. Siccome ad ogni nuovo nodo aggiunto al nostro insieme Z dovremo far ripartire la procedura di ricerca del nodo minimizzante, necessitiamo di una struttura dati che ci permetta di estrarre il valore minimo e di inserirne di nuovi in modo efficiente: utilizziamo dunque un **Min-Heap**.

```
    def algoritmoDijkstra(G, w, x):
      dist = [float('inf')] * n
      dist[x] = 0
      P = [-1] * n
      P[x] = x
      H = Heap()

      # 1. Inserimento degli archi uscenti dalla sorgente
      for (x, v) in E(G):
        H.insert([w((x, v)), (x, v)])

      # 2. Estrazione progressiva e aggiornamento dei nodi
      for i in range(n - 1):
        (u, v) = H.extract_min()

      if P[u] == -1:
        new = u
        old = v
      else:
        new = v
        old = u

      P[new] = old
      dist[new] = dist[old] + w((u, v))

      # 3. Aggiornamento dello stato dell'heap con i nuovi
         archi
```

```
27        for y in new.neighbor:
28          if P[y] == -1:
29            H.insert([dist[new] + w((new, y)), (new, y)])
30          else:
31            H.remove((new, y))
32
33        return P
```

Listing 1.15: Algoritmo di Dijkstra con Min-Heap

Spiegazione algoritmo 1.11.1: Algoritmo di Dijkstra

- **Inizializzazione delle strutture dati**:
  - `dist`: Array contenente le distanze minime calcolate a partire dalla sorgente x, inizializzato a ∞ per tutti i vertici tranne per la sorgente dove $\text{dist}[x] = 0$.
  - `P`: Vettore dei padri impiegato per la ricostruzione del cammino minimo. Inizialmente ha tutti i valori a -1, tranne $\text{P}[x] = x$.
  - `H`: Struttura dati Min-Heap contenente man mano gli archi uscenti dall'insieme logico dei nodi correntemente visitati Z.
- **Fase 1 – Popolamento iniziale dello Heap**: Con il ciclo `for (x, v) in E(G)`, aggiungiamo allo heap tutti gli archi uscenti dalla sorgente x. Essendo inizialmente $Z = \{x\}$, gli unici archi uscenti da Z sono proprio quelli che partono da x.
- **Fase 2 – Estrazione ed espansione del cammino minimo**: Iteriamo per $n - 1$ volte al fine di includere tutti i nodi del grafo nell'insieme Z:
  - `(u, v) = H.extract_min()`: Estraiamo dallo heap l'arco che presenta la distanza minima dal bordo dell'insieme Z verso l'esterno.
  - **Identificazione del nodo esterno** (`if P[u] == -1`): Effettuiamo un controllo per verificare quale dei due vertici dell'arco si trova dentro Z e quale si trova fuori. Utilizziamo il vettore dei padri P come rappresentazione implicita dell'insieme Z (se $\text{P}[u] == -1$, allora $u \notin Z$). Assegniamo il valore a `new` per il nodo fuori da Z e ad `old` per il nodo dentro Z.

  - **Aggiornamento dei cammini**: Il padre di `new` diventerà `old` (P[new] = old), e la distanza viene aggiornata come dist[new] = dist[old] + $w((u, v))$, in diretta conseguenza delle proprietà sui cammini minimi.

- **Fase 3 – Aggiornamento degli archi uscenti**: Esaminiamo tutti i vicini y del nodo appena aggiunto `new`:
  - Se $y \notin Z$ (ovvero P[y] == -1), inseriamo il nuovo arco uscente nello heap valorizzato con la chiave dist[new] + $w((\text{new}, y))$.
  - Se invece $y \in Z$, rimuoviamo l'arco (new, y) dallo heap mediante `H.remove((new, y))`, poiché collegherebbe due nodi già appartenenti a Z e quindi già considerati.

Analizzando la complessità si ottiene che:

- Il `for` in (1) esegue un'operazione di costo $\log(m)$ per un totale di al più m volte (caso in cui da x partono tutti gli archi del grafo, sarà $n - 1$). Il costo totale è dunque $O(m \log m)$. Volendo però esprimerlo in funzione di n, e ricordandoci che stiamo aggiungendo tutti gli archi uscenti da x che sono al più $n - 1$, il costo di questo ciclo diventa $O(n \log n)$.
- Il ciclo in (2) si ripete banalmente n volte e al suo interno:
  - Si estrae il minimo dall'heap che costa, alla fine dell'algoritmo, $O(\log m)$, dove stavolta m sono tutti gli archi del grafo, dunque al più n^2. Otteniamo dunque $O(\log n^2) = O(2 \log n) = O(\log n)$.
  - Il ciclo in (3) è descrivibile dalla sommatoria

$$\sum_{e \in E(G)} \log(n) = O(m \log n)$$

Il costo totale:

$$O(n \log n) + O(m \log n) = \mathbf{O((n + m) \log n)}$$

2

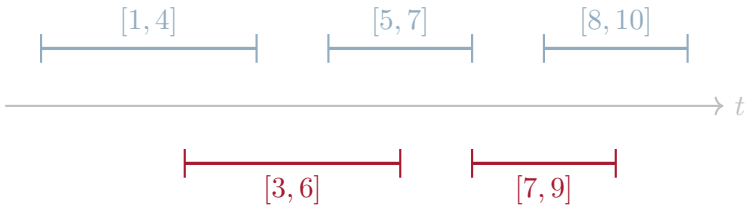
Algoritmi Greedy

Gli **algoritmi greedy** si occupano di risolvere problemi di ottimizzazione, cercando di individuare la soluzione globale migliore attraverso una serie di scelte locali ottime ad ogni passo.

Esercizio 2.0.1: Problema della selezione degli intervalli

Dato in input un insieme di n intervalli del tipo $[a_i, b_i]$, per $i = 1, \ldots, n$, l'obiettivo è trovare un sottoinsieme di intervalli *mutuamente disgiunti* (compatibili) di *cardinalità massima*.

A titolo esemplificativo, consideriamo la seguente istanza del problema:



Dall'immagine si nota come una scelta parziale corretta possa portare a soluzioni differenti; per questo problema, l'insieme $\{[1, 4], [5, 7], [8, 10]\}$ costituisce una soluzione totale ottima di cardinalità 3.

2.0.1 Strategie di ordinamento per l'algoritmo Greedy

Per applicare la tecnica greedy occorre stabilire un criterio di ordinamento per gli intervalli.

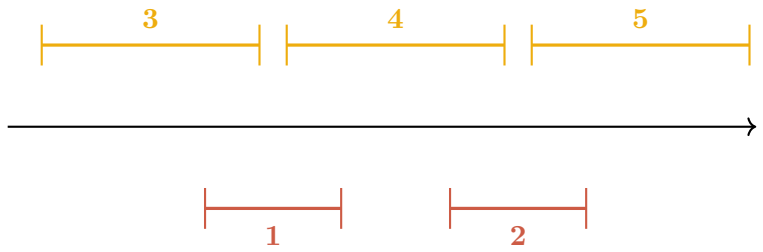
Idea 1: Ordinamento per estremo inferiore. Se ordiniamo gli intervalli in base al loro estremo inferiore a_i, la soluzione trovata è corretta ma *non è sempre ottima*. Scegliendo infatti come primo intervallo uno molto ampio, rischiamo di eliminare molti piccoli intervalli ad esso sovrapposti ma reciprocamente compatibili.

Controesempio:

$$I_1 = [1, 100], \quad I_2 = [2, 3], \quad I_3 = [4, 5], \ldots$$

L'algoritmo selezionerebbe l'intervallo $[1, 100]$, scartando tutti i successivi ed ottenendo una soluzione di cardinalità 1, contro una soluzione ottima di cardinalità molto più elevata.

Idea 2: Ordinamrnto per ampiezza degli intervalli. Anche la seconda strategia *non garantisce l'ottimalità*. **Controesempio:** consideriamo il seguente insieme di 5 intervalli.



Se ordiniamo per ampiezza crescente, l'algoritmo sceglie prima gli intervalli 1 e 2 (aventi lunghezza inferiore). Tuttavia, l'inclusione di 1 e 2 rende incompatibili tutti e tre gli intervalli superiori.

Idea 3: Ordinamento per estremo superiore. In questo caso, l'ordinamento crescente con chiave l'estremo superiore di ogni intervallo risulta essere la scelta corretta per ottenere la soluzione ottimale al problema. La dimostrazione di correttezza verrà fatta in seguito alla presentazione dell'algoritmo, ma si può verificare come sia impossibile trovare un controesempio.

```
1    def intervalli(sets):
2      sol = set()
3      # ordinare sets in ordine crescente in base all'estremo
           superiore
4
5      for i in range(n):
6        if sets[i] non interseca alcun S in sol:
7          sol.add(sets[i])
```

```
8
9       return sol
```

Listing 2.1: Algoritmo Greedy per la selezione di intervalli

Spiegazione algoritmo 2.0.1: Selezione di intervalli

1. Inizializza un insieme vuoto `sol` per memorizzare la soluzione finale.
2. Ordina gli intervalli dell'insieme `sets` in ordine crescente in base al loro estremo superiore.
3. Per ogni intervallo `sets[i]`, controlla se esso interseca qualche intervallo $S \in$ `sol`. In caso negativo, l'intervallo viene aggiunto a `sol`.
4. Restituisce l'insieme `sol` degli intervalli selezionati.